

APRENDIZAJE POR DEMOSTRACIÓN DE UN BRAZO ROBÓTICO A PARTIR DE GESTOS HUMANOS

TRABAJO FIN DE GRADO
PARA LA TITULACIÓN DE
GRADO EN ROBÓTICA

AUTOR:
SERGIO GONZÁLEZ RODRÍGUEZ

TUTORES:
Dr. JUAN ANTONIO CORRALES RAMÓN
Dpto. de Electrónica y Computación
Dr. XOSE RAMÓN FERNÁNDEZ VIDAL
Dpto. de Física aplicada

CONVOCATORIA:
Febrero de 2026

UNIVERSIDADE DE SANTIAGO DE COMPOSTELA

TRABAJO FIN DE GRADO

APRENDIZAJE POR DEMOSTRACIÓN DE UN BRAZO ROBÓTICO A PARTIR DE GESTOS HUMANOS

AUTOR:
SERGIO GONZÁLEZ
RODRÍGUEZ

TUTORES:
Dr. JUAN ANTONIO
CORRALES RAMÓN
Dpto. de Electrónica y Computación

Dr. XOSE RAMÓN
FERNANDEZ VIDAL
Dpto. de Física aplicada

*Este trabajo se presenta para cumplir con los requisitos reglamentarios
requeridos para lograr el Grado en Robótica*

na

Escola Politécnica Superior de Enxeñaría.

Fecha de la Convocatoria: Febrero de 2026



ESCOLA POLITÉCNICA SUPERIOR
DE ENXEÑARÍA

Autorización

Yo, SERGIO GONZÁLEZ RODRÍGUEZ, alumno de la titulación del Grado en Robótica de la **Escola Politécnica Superior de Enxeñaría** de la **Universidade de Santiago de Compostela**, con DNI 79.150.965 F, autorizo la ubicación de la siguiente copia de mi **Traballo Fin de Grao** en la biblioteca de Intercentros para que pueda ser consultada por las personas que lo deseen.

Firmado: Sergio González Rodríguez

Fecha: 30/01/2026

Solo podemos ver un poco del futuro, pero lo suficiente para darnos cuenta de que hay mucho por hacer.

Alan Turing

Resumo

APRENDIZAJE POR DEMOSTRACIÓN DE UN BRAZO ROBÓTICO A PARTIR DE GESTOS HUMANOS

por
SERGIO GONZÁLEZ RODRÍGUEZ

Palabras Chave: *Robótica Colaborativa, Aprendizaxe por Demostración, Mapeo Cinemático, Primitivas de Movemento Dinámico, Estimación de Pose Humana*

Na industria moderna, a robótica colaborativa xoga un papel cada vez máis relevante, permitindo a automatización de tarefas complexas en contornas compartidas con humanos. Con todo, a programación tradicional destes robots segue a ser unha barreira significativa, xa que require coñecementos técnicos especializados e consúme un tempo considerable, dificultando a súa adaptación áxil a novos procesos. As simulacións, aínda que útiles, a miúdo non logran capturar a complexidade e variabilidade do mundo real.

Unha alternativa potente é a Aprendizaxe por Demostración (LfD), un paradigma que busca transferir unha habilidade directamente dende un experto humano (sistema A) a un robot (sistema B). O obxectivo é que o robot poida aprender e reproducir tarefas de manipulación complexas a través da simple observación, superando as limitacións da programación explícita e aproveitando o coñecemento implícito do operario.

Este traballo céntrase no desenvolvemento e na validación dun sistema completo de LfD para esta transferencia de coñecemento humano-robot. Impleméntase un pipeline que integra un sistema de visión estéreo para a captura 3D da demostración, un novidoso mapeo cinemático que traduce a pose humana ao espazo articular do robot resolvendo as diferenzas morfolóxicas, e un modelo de aprendizaxe baseado en Primitivas de Movemento Dinámico (DMPs) que xeneraliza a habilidade a partir de múltiples exemplos. A clara implicación deste sistema é a capacidade de ensinar tarefas complexas a un robot de forma rápida e intuitiva, sentando as bases para unha automatización máis flexible e accesible.

Resumen

APRENDIZAJE POR DEMOSTRACIÓN DE UN BRAZO ROBÓTICO A PARTIR DE GESTOS HUMANOS

por
SERGIO GONZÁLEZ RODRÍGUEZ

Palabras Clave: *Robótica Colaborativa, Aprendizaje por Demostración, Mapeo Cinemático, Primitivas de movimiento dinámico*

En la industria moderna, la robótica colaborativa juega un papel cada vez más relevante, permitiendo la automatización de tareas complejas en entornos compartidos con humanos. Sin embargo, la programación tradicional de estos robots sigue siendo una barrera significativa, ya que requiere conocimientos técnicos especializados y consume un tiempo considerable, dificultando su adaptación ágil a nuevos procesos. Las simulaciones a menudo no logran capturar la complejidad y variabilidad del mundo real.

Una alternativa potente es el Aprendizaje por Demostración (LfD), una tecnología que busca transferir una habilidad directamente desde un experto humano (sistema A) a un robot (sistema B). El objetivo es que el robot pueda aprender y reproducir tareas de manipulación complejas a través de la observación, superando las limitaciones de la programación explícita y aprovechando el conocimiento implícito del operario.

Este trabajo se centra en el desarrollo y la validación de un sistema completo de LfD para esta transferencia de conocimiento humano-robot. Se implementa un pipeline que integra un sistema de visión estéreo para la captura 3D de la demostración, un novedoso mapeo cinemático que traduce la pose humana al espacio articular del robot resolviendo las diferencias morfológicas, y un modelo de aprendizaje basado en Primitivas de Movimiento Dinámico (DMPs) que generaliza la habilidad a partir de múltiples ejemplos. La clara implicación de este sistema es la capacidad de enseñar tareas complejas a un robot de forma rápida e intuitiva, sentando las bases para una automatización más flexible y accesible.

Abstract

APRENDIZAJE POR DEMOSTRACIÓN DE UN BRAZO ROBÓTICO A PARTIR DE GESTOS HUMANOS

by
SERGIO GONZÁLEZ RODRÍGUEZ

Keywords: *Collaborative Robotics, Learning from Demonstration, Kinematic Mapping, Dynamic Movement Primitives, Human Pose Estimation*

In modern industry, collaborative robotics plays an increasingly relevant role, enabling the automation of complex tasks in shared environments with humans. However, the traditional programming of these robots remains a significant barrier, as it requires specialized technical expertise and consumes considerable time, hindering their agile adaptation to new processes. Simulations, while useful, often fail to capture the complexity and variability of the real world.

A powerful alternative is Learning from Demonstration (LfD), a paradigm that seeks to transfer a skill directly from a human expert (system A) to a robot (system B). The objective is for the robot to learn and reproduce complex manipulation tasks through simple observation, overcoming the limitations of explicit programming and leveraging the operator's implicit knowledge.

This work focuses on the development and validation of a complete LfD system for this human-to-robot knowledge transfer. A pipeline is implemented that integrates a stereo vision system for 3D capture of the demonstration, a novel kinematic mapping that translates the human pose into the robot's joint space while resolving morphological differences, and a learning model based on Dynamic Movement Primitives (DMPs) that generalizes the skill from multiple examples. The clear implication of this system is the ability to teach complex tasks to a robot quickly and intuitively, laying the groundwork for more flexible and accessible automation.

Agradecimientos

Agradecer a mis tutores Juan Antonio y Xose Ramón por ayudarme a llevar a cabo este TFG, que pone el broche final al aprendizaje de todos estos años.

Gracias a mis compañeros y amigos de la carrera, que me ayudaron a no perder el foco y que las dificultades del camino fueran más ligeras.

Gracias a mi familia por apoyarme y acompañarme desde la distancia.

Gracias a Galicia y los gallegos, por hacerme sentir acogido tan lejos de casa.

Finalmente, gracias a todas las personas que me encontré en este camino, que han aportado de manera técnica o emocional a este trabajo, logrando que sea el cierre perfecto de esta etapa universitaria.

Índice General

Autorización	III
Resumo	VII
Resumen	IX
Abstract	XI
Agradecimientos	XIII
1. Introducción al tema del TFG	1
1.1. Introducción	1
1.2. Motivación	2
1.3. Objetivos	2
1.4. Estructura de la memoria	2
2. Estado del Arte	5
2.1. Fundamentos de Manipulación Robótica	5
2.1.1. Cinemática Directa y Parámetros Denavit-Hartenberg	5
2.1.2. Cinemática Inversa y Redundancia	6
2.2. Visión Artificial y Percepción 3D	6
2.2.1. Modelo de Cámara y Calibración Extrínseca	7
2.2.2. Detección de Pose Humana: MediaPipe vs OpenPose	7
2.2.3. Triangulación 3D y Manejo de Oclusiones	8
2.3. Detección de Objetos mediante Aprendizaje Profundo	9
2.3.1. Redes Neuronales Convolucionales (CNNs)	10
2.3.2. Arquitectura YOLO (You Only Look Once)	10
2.4. Aprendizaje por Demostración (LfD)	11
2.4.1. Taxonomía de las Estrategias de Enseñanza	11
2.4.2. Limitaciones de la Grabación Directa	12
2.5. Primitivas de Movimiento Dinámico (DMPs)	12
2.5.1. Formulación Matemática	12
El Sistema Canónico	12
El Sistema de Transformación	12
2.5.2. Aprendizaje del Término de Fuerza	13
2.6. Análisis Comparativo con Estrategias Existentes	14
2.6.1. End-to-End vs. Pipeline Modular	14
2.6.2. Sistemas con Marcadores vs. Markerless	14
3. Sistemas de percepción	17
3.1. Arquitectura del Sistema de Percepción	17

3.1.1.	Hardware y Disposición de Sensores	17
3.1.2.	Stack Tecnológico y Flujo de Datos	18
3.2.	Calibración del Sistema Estéreo	18
3.2.1.	Calibración Extrínseca Cámara-Robot	19
3.3.	Detección y Reconocimiento en 2D	20
3.3.1.	Estimación de Pose Humana (MediaPipe)	20
3.3.2.	Detección de Objetos (YOLOv8 Personalizado)	20
3.4.	Reconstrucción 3D y Lógica de Escena	22
3.4.1.	Triangulación Estéreo	22
3.4.2.	Máquina de Estados Basada en Eventos	22
4.	Control	25
4.1.	El Desafío de la Correspondencia Morfológica	25
4.2.	Algoritmo de Mapeo Cinemático	25
4.2.1.	Transformación y Reflexión Morfológica	25
4.2.2.	Extracción de Ángulos Relativos	26
4.2.3.	Mapeo Lineal al Espacio Articular del Robot	27
4.3.	Validación y Simulación Cinemática	27
4.4.	Pipeline de Aprendizaje de Primitivas (DMP Learning)	28
4.4.1.	Preprocesamiento y Normalización Temporal	28
4.4.2.	Extracción de la Trayectoria Maestra (Averaging)	28
4.4.3.	Entrenamiento y Serialización del Modelo	29
5.	Experimentación	31
5.1.	Arquitectura de Despliegue	31
5.2.	Estrategia de Control y Ejecución	31
5.2.1.	Capa de Seguridad y Límites	31
5.2.2.	Control Híbrido para la Fase de Manipulación	32
5.3.	Evaluación del Subsistema de Percepción	32
5.3.1.	Métricas de Entrenamiento (YOLOv8)	32
5.3.2.	Análisis de Latencia y Tiempo de Cómputo	34
5.4.	Resultados de la Ejecución Robótica	34
5.4.1.	Fase 1: Aproximación y Generalización	35
5.4.2.	Fase 2: Interacción y Agarre	35
5.4.3.	Fase 3: Manipulación (Desmolde)	35
5.5.	Discusión y Limitaciones	36
6.	Conclusiones y Líneas de Trabajo Futuro	39
6.1.	Conclusiones Generales	39
6.2.	Conclusiones Técnicas	39
6.2.1.	Sobre el Sistema de Percepción	39
6.2.2.	Sobre el Mapeo Cinemático y Control	39
6.2.3.	Sobre la Generalización con DMPs	40
6.3.	Líneas de Trabajo Futuro	40
A.	Repositorio de Código y Recursos Digitales	41
	Bibliografía	43

Índice de figuras

2.1. Asignación de sistemas de coordenadas en el robot UR3 siguiendo el convenio Denavit-Hartenberg. Imagen extraída de [5].	6
2.2. Nube de puntos hecha con Cámara Realsense en laboratorio, mostrando la capacidad de la RealSense, aunque con alto coste computacional. [6].	7
2.3. Captura de pose comparando OpenPose y Mediapipe. Imagen adaptada de [9].	8
2.4. Principio geométrico de la triangulación estéreo. La posición del objeto en 3D se recupera mediante la intersección de los rayos proyectados desde los planos de imagen de dos cámaras calibradas. Imagen adaptada de [11].	9
2.5. Ejemplo de red con múltiples capas convolucionales. Se aplican filtros a las imágenes de entrenamiento con distintas resoluciones, y la salida resultante de convolucionar cada imagen se emplea como entrada para la siguiente capa. Imagen extraída de [13].	10
2.6. Arquitectura del modelo de detección YOLO. El sistema divide la imagen de entrada en una rejilla $S \times S$ y, para cada celda, predice simultáneamente las cajas delimitadoras (bounding boxes) y las probabilidades de clase. Imagen extraída de [14].	11
2.7. Funcionamiento del término de fuerza. Arriba: Activación secuencial de las funciones de base Gaussianas (ψ_i). Abajo: Reconstrucción de la forma de la trayectoria mediante la suma ponderada. Imagen adaptada de [17].	14
2.8. Ejemplo del sistema de captura marker-based, para ilustrar la cantidad de elementos usados en este tipo de sistemas. Imagen adaptada de [21].	15
3.1. Setup inicial de las primeras pruebas en laboratorio. Se observa el sistema YOLO detectando objetos y MediaPipe detectando la pose del operario.	17
3.2. Diagrama de flujo del sistema de percepción. El sistema procesa paralelamente las detecciones 2D para posteriormente triangular la posición 3D y alimentar la máquina de estados.	18
3.3. Captura de pantalla del proceso de detección de esquinas en el patrón de ajedrez para la calibración.	19
3.4. Configuración experimental para la calibración extrínseca cámara-base. El robot UR3e posiciona con precisión el patrón de ajedrez en el campo de visión de las cámaras RealSense. Este proceso permite obtener la matriz de transformación que relaciona el sistema de coordenadas de visión con la base del robot.	20

3.5.	Correlograma de etiquetas del dataset de entrenamiento. Se muestra la distribución espacial (x, y) y dimensional ($width, height$) de las instancias anotadas. La concentración de datos en el eje $y > 0,5$ refleja la disposición física del experimento, con los objetos situados sobre la mesa de trabajo en la mitad inferior del encuadre.	21
3.6.	Muestra del conjunto de validación con las etiquetas de verdad terreno (<i>ground truth</i>) utilizadas para el reentrenamiento de YOLOv8n.	22
3.7.	Visualización en tiempo real del sistema de percepción completo. A los lados: Vistas de cámara 1 y 2 con detección de pose (MediaPipe) y objetos (YOLO). Centro: Reconstrucción 3D del esqueleto mediante triangulación estéreo en Open3D. Nótese el indicador de estado actual ("Rotating") en la vista izquierda, resultado de la máquina de estados.	23
4.1.	Entorno de simulación en RViz, se ilustra que si aplicamos el DMP sin hacer la reflexión morfológica, quedaría debajo del suelo en simulación, lo que corresponde a la mesa de apoyo.	26
4.2.	Visualización del entorno de validación desarrollado. En azul: esqueleto humano capturado y transformado. En verde: reconstrucción cinemática del robot UR3e ejecutando los ángulos calculados. Nótese la configuración "codo arriba" del robot frente a la "codo abajo" del humano, resultado de la reflexión morfológica.	27
4.3.	Trayectorias articulares generadas por el modelo DMP para la fase de aproximación. Se observa la suavidad de las curvas (C^2 continuo) y la convergencia asintótica al objetivo, eliminando el ruido de alta frecuencia presente en la captura humana original. Cada color representa el perfil de posición de una articulación del robot (q_1 a q_6).	29
5.1.	Evolución de las funciones de pérdida y métricas de desempeño durante el entrenamiento del detector. Fila superior: Pérdidas en conjunto de entrenamiento y métricas globales. Fila inferior: Pérdidas en conjunto de validación y métricas mAP. Se observa una convergencia estable a partir de la época 40.	33
5.2.	Matriz de confusión normalizada del modelo YOLOv8n re-entrenado. Se observa una alta precisión en la clase crítica <code>doll_head</code> (95 %), mientras que la clase <code>doll_body</code> presenta una mayor tasa de falsos negativos debido a oclusiones parciales durante la manipulación.	34
5.3.	Secuencia de ejecución en el robot real UR3e. (A - Izquierda arriba) Posición inicial de reposo. (B - Izquierda abajo) Aproximación guiada por DMP. (C - Derecha arriba) Agarre del objeto mediante activación de E/S digitales. (D - Derecha abajo) Ejecución de la rotación pura de muñeca para el desmolde y separación de la cabeza.	36

Índice de Tablas

2.1. Parámetros de Denavit-Hartenberg para el robot UR3e.	5
3.1. Definición de la Máquina de Estados para la segmentación de la tarea.	23

Lista de Abreviaciones

LfD	L earning from D emonstration
DMP	D ynamic M ovement P rimitives
ROS	R obotic O perative S ystem
YOLO	Y ou O only L ook O nce
RGB-D	R ed G reen B lue D epth
DLT	D irect L inear T ransformation
CNN	C onvolutional N eural N etwork
ReLU	R ectified L inear U nit
SiLU	S igmoid L inear U nit
PD	P roportional D erivative

Dedicado a mi familia y amigos, gracias por todo ...

1 Introducción al tema del TFG

1.1. Introducción

En la industria actual, la robótica colaborativa está transformando los procesos de manufactura al permitir que los robots trabajen junto a los operarios humanos. Esta colaboración abre la puerta a una automatización más flexible, pero a la vez expone una de sus mayores limitaciones: la programación. Los métodos tradicionales para enseñar tareas a los robots, como la programación textual o el guiado manual, son procesos lentos que requieren personal especializado y que no son adecuados para entornos de producción ágiles. Es por esto que dominar métodos de programación más intuitivos es de gran interés para la industria.

Una de las estrategias más prometedoras para dotar de nuevas habilidades a los robots es el Aprendizaje por Demostración (*Learning from Demonstration*, LfD). El LfD es un paradigma del aprendizaje automático en el que un robot aprende una política de comportamiento a través de la observación de un experto, generalmente un humano [1]. Aunque este enfoque es conceptualmente muy potente, su aplicación en entornos reales presenta desafíos significativos. El principal de ellos es el conocido como *problema de la correspondencia* (correspondence problem), que se refiere a la dificultad de traducir correctamente las acciones observadas en el demostrador a las acciones que debe ejecutar el robot [2].

El problema de la correspondencia surge porque el profesor (el humano) y el aprendiz (el robot) no son idénticos. Desde el punto de vista de la percepción, el humano observa el mundo a través de su visión binocular y su propiocepción, mientras que el robot lo hace a través de su sistema de cámaras, lo que genera diferentes representaciones del espacio de estados. De forma aún más crítica, existen diferencias en el espacio de acciones: el brazo humano posee más de siete grados de libertad y una morfología colgante, mientras que un brazo robótico como el UR3e tiene seis grados de libertad, límites articulares definidos y una base fija. Traducir directamente los movimientos de uno a otro es, por tanto, una tarea no trivial debido a estas restricciones cinemáticas y dinámicas [3].

Este trabajo se centra en desarrollar una solución completa a este problema para una tarea de manipulación. Se propone un pipeline que aborda los desafíos de la correspondencia mediante: (1) un sistema de visión estéreo para la captura 3D robusta de la pose humana, y (2) un pipeline de mapeo cinemático que traduce la cinemática humana a la del robot en el espacio articular, resolviendo las diferencias morfológicas. Finalmente, se utiliza este mapeo para generar trayectorias de entrenamiento para un modelo de Primitivas de Movimiento Dinámico (DMPs), permitiendo al robot aprender y generalizar la habilidad demostrada.

1.2. Motivación

La motivación principal de este trabajo surge en el marco del proyecto de investigación europeo **ROBOTA-SUDOE**, cuyo objetivo es desarrollar nuevas aplicaciones para robots colaborativos en sectores estratégicos. Este TFG aborda una de sus sub-tareas: el ensamblado y desmolde de una muñeca de juguete, una tarea de manipulación que, aunque acotada, encapsula desafíos representativos como la aproximación precisa, el agarre, la rotación coordinada y la separación de piezas.

El plan de instrumentación inicial del proyecto contemplaba un hardware de última generación (cámaras ZED X Mini y un robot UR5). Debido a contratiempos en la entrega de dicho material, este trabajo afrontó el desafío de adaptar la metodología a un hardware alternativo, utilizando cámaras Intel RealSense y un brazo robótico UR3e. Por tanto, este proyecto se justifica como una **prueba de concepto y un prototipo a escala** que desarrolla y valida la totalidad del pipeline de software de LfD, sentando una base metodológica fundamental y directamente transferible para la implementación final del proyecto principal.

1.3. Objetivos

Para dar respuesta al problema planteado y validar la metodología propuesta, se definen los siguientes objetivos:

- **Objetivo Principal:** Desarrollar y validar un sistema completo de Aprendizaje por Demostración (LfD) que permita a un brazo robótico UR3e aprender una tarea de manipulación a partir de la observación 3D de un operador humano.
- **Objetivos Parciales:**
 1. Implementar un sistema de visión estéreo para la captura robusta de demostraciones humanas en 3D.
 2. Entrenar y desplegar un detector de objetos a medida basado en YOLOv8 para la localización precisa de las piezas de la tarea.
 3. Diseñar un pipeline de mapeo cinemático para traducir el movimiento del operario al espacio articular del robot, resolviendo las diferencias morfológicas.
 4. Entrenar un modelo generativo de movimiento basado en Primitivas de Movimiento Dinámico (DMPs) para aprender una representación generalizada de la habilidad.
 5. Integrar el modelo aprendido con el ecosistema ROS para ejecutar la tarea de forma autónoma en el robot real y simulado.

1.4. Estructura de la memoria

El presente documento se estructura en los siguientes capítulos que detallan el proceso de investigación y desarrollo llevado a cabo:

Capítulo 2: Estado del arte. Establece el marco teórico, revisando la literatura actual sobre el Aprendizaje por Demostración, las técnicas de visión artificial para

captura de movimiento sin marcadores y los fundamentos matemáticos de las Primitivas de Movimiento Dinámico (DMPs).

Capítulo 3: Sistema de percepción. Detalla el desarrollo del sistema de visión estéreo y el entrenamiento del detector neuronal basado en YOLOv8 para la localización de objetos.

Capítulo 4: Control. Aborda el núcleo del problema de correspondencia. Se explica el pipeline de mapeo cinemático diseñado para traducir la pose humana al robot UR3e y la formulación del algoritmo de aprendizaje.

Capítulo 5: Experimentación. Presenta los experimentos realizados tanto en simulación como en el entorno real, validando la precisión del mapeo y la capacidad de generalización del sistema.

Capítulo 6: Conclusiones. Recoge las conclusiones finales, discute las limitaciones encontradas y propone líneas de trabajo futuro.

2 Estado del Arte

Este capítulo analiza las tecnologías y metodologías que fundamentan este trabajo. Se revisan tres pilares fundamentales: los modelos matemáticos para la representación de trayectorias robóticas, las estrategias de Aprendizaje por Demostración y la visión artificial aplicada a la captura de movimiento humano.

2.1. Fundamentos de Manipulación Robótica

Para comprender la problemática del mapeo de movimiento humano a robot, es necesario establecer primero las bases matemáticas que rigen el movimiento del manipulador utilizado, el UR3e.

2.1.1. Cinemática Directa y Parámetros Denavit-Hartenberg

La cinemática directa permite calcular la posición y orientación del efector final del robot en el espacio cartesiano, dados los valores de sus articulaciones $q = [q_1, q_2, \dots, q_n]$. Para robots seriales como el UR3e, la convención estándar para sistematizar esta relación es la de Denavit-Hartenberg (D-H).

Según la documentación oficial del fabricante [4]:

CUADRO 2.1: Parámetros de Denavit-Hartenberg para el robot UR3e.

Link (i)	a_i (m)	α_i (rad)	d_i (m)	θ_i
1	0	$\pi/2$	0,1519	θ_1
2	-0,24365	0	0	θ_2
3	-0,21325	0	0	θ_3
4	0	$\pi/2$	0,11235	θ_4
5	0	$-\pi/2$	0,08535	θ_5
6	0	0	0,08190	θ_6

Esta convención establece que la transformación entre dos articulaciones consecutivas $i - 1$ e i viene dada por la matriz de transformación homogénea A_i :

$$A_i = \begin{bmatrix} \cos \theta_i & -\sin \theta_i \cos \alpha_i & \sin \theta_i \sin \alpha_i & a_i \cos \theta_i \\ \sin \theta_i & \cos \theta_i \cos \alpha_i & -\cos \theta_i \sin \alpha_i & a_i \sin \theta_i \\ 0 & \sin \alpha_i & \cos \alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (2.1)$$

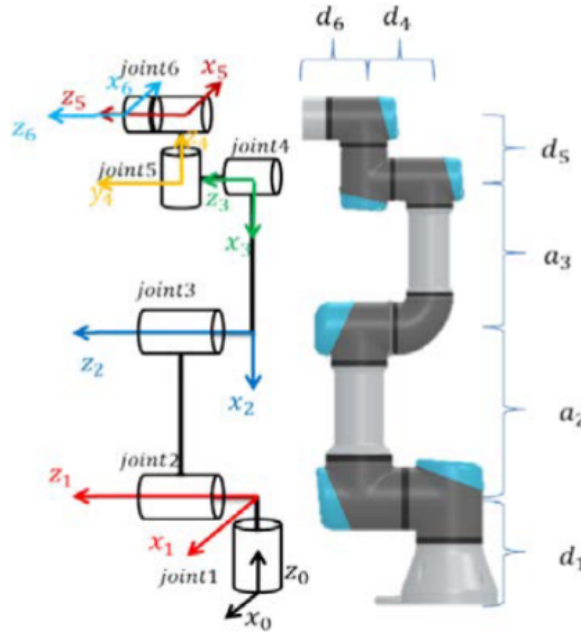


FIGURA 2.1: Asignación de sistemas de coordenadas en el robot UR3 siguiendo el convenio Denavit-Hartenberg. Imagen extraída de [5].

Donde $\theta_i, d_i, a_i, \alpha_i$ son los cuatro parámetros que definen la geometría del eslabón. En el caso de este trabajo, el conocimiento preciso de la cinemática directa es fundamental para validar que las trayectorias generadas por el algoritmo de aprendizaje se encuentran dentro del espacio de trabajo alcanzable del robot.

2.1.2. Cinemática Inversa y Redundancia

El problema inverso, obtener la configuración articular q necesaria para alcanzar una pose cartesiana T , es de mayor complejidad. Mientras que el brazo humano es redundante (posee 7 o más grados de libertad para posicionar la mano en el espacio 3D), el UR3e posee 6 grados de libertad. Esto implica que, para una pose dada, existe un número finito de soluciones matemáticas (normalmente hasta 8 configuraciones posibles, ej: “codo arriba” vs “codo abajo”).

El algoritmo de mapeo propuesto en el Capítulo 4 abordará esta limitación, seleccionando la solución que minimice la discrepancia con la postura natural observada en el operario humano.

2.2. Visión Artificial y Percepción 3D

La percepción del entorno y del operario es el primer eslabón crítico en el sistema de Aprendizaje por Demostración. Aunque existen sensores RGB-D capaces de entregar mapas de profundidad densos (como la tecnología Intel RealSense), este trabajo opta por un enfoque de **visión estéreo pasiva basada en triangulación de puntos clave (keypoints)**.

Esta decisión de diseño se fundamenta en la eficiencia computacional: procesar nubes de puntos densas ($> 300,000$ puntos por frame, Figura 2.2) introduce latencias

incompatibles con una interacción fluida en tiempo real. En su lugar, se propone un esquema que primero reduce la dimensionalidad mediante la detección de esqueletos en 2D y posteriormente reconstruye la geometría 3D mediante triangulación estéreo, combinando robustez y velocidad.



FIGURA 2.2: Nube de puntos hecha con Cámara Realsense en laboratorio, mostrando la capacidad de la RealSense, aunque con alto coste computacional. [6].

2.2.1. Modelo de Cámara y Calibración Extrínseca

Para relacionar la información visual con el mundo físico, se asume el modelo de cámara *pinhole*. Un punto en el espacio $P = [X, Y, Z]^T$ se proyecta en el plano de imagen $p = [u, v]^T$ mediante la matriz de parámetros intrínsecos K :

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \begin{bmatrix} R & t \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \quad (2.2)$$

Donde R y t representan la rotación y traslación de la cámara respecto al sistema de coordenadas global.

En una configuración estéreo con dos cámaras (Cámara A y Cámara B), es vital conocer la transformación rígida relativa entre ellas. Esta relación se obtiene mediante un proceso de calibración extrínseca utilizando un patrón de ajedrez (chessboard) y el algoritmo de Zhang [7]. Esto nos proporciona la matriz de transformación homogénea ${}^A T_B$, que permite expresar cualquier rayo óptico de la cámara B en el sistema de referencia de la cámara A.

2.2.2. Detección de Pose Humana: MediaPipe vs OpenPose

La extracción de características se realiza en el espacio 2D. En la literatura existen dos referentes principales: OpenPose y MediaPipe. Mientras que OpenPose sigue un enfoque *bottom-up* (detecta todas las partes del cuerpo en la imagen y luego las asocia a personas), MediaPipe [8] utiliza un enfoque *top-down* optimizado para CPU.

Estudios recientes comparativos en captura de movimiento con recursos limitados [9] han demostrado que, si bien OpenPose puede ofrecer mayor robustez en multitudes, MediaPipe es significativamente superior en términos de eficiencia computacional y facilidad de despliegue en hardware convencional, lo que lo convierte en la opción óptima para aplicaciones de tiempo real como la propuesta en este trabajo.

Para este TFG se ha seleccionado **MediaPipe** debido a su mayor estabilidad temporal (menor *jitter* en las detecciones) y su arquitectura ligera, lo que libera recursos computacionales para los algoritmos de control y aprendizaje del robot. MediaPipe entrega, para cada imagen i , un conjunto de coordenadas pixel (u_j, v_j) correspondientes a las j articulaciones del cuerpo.

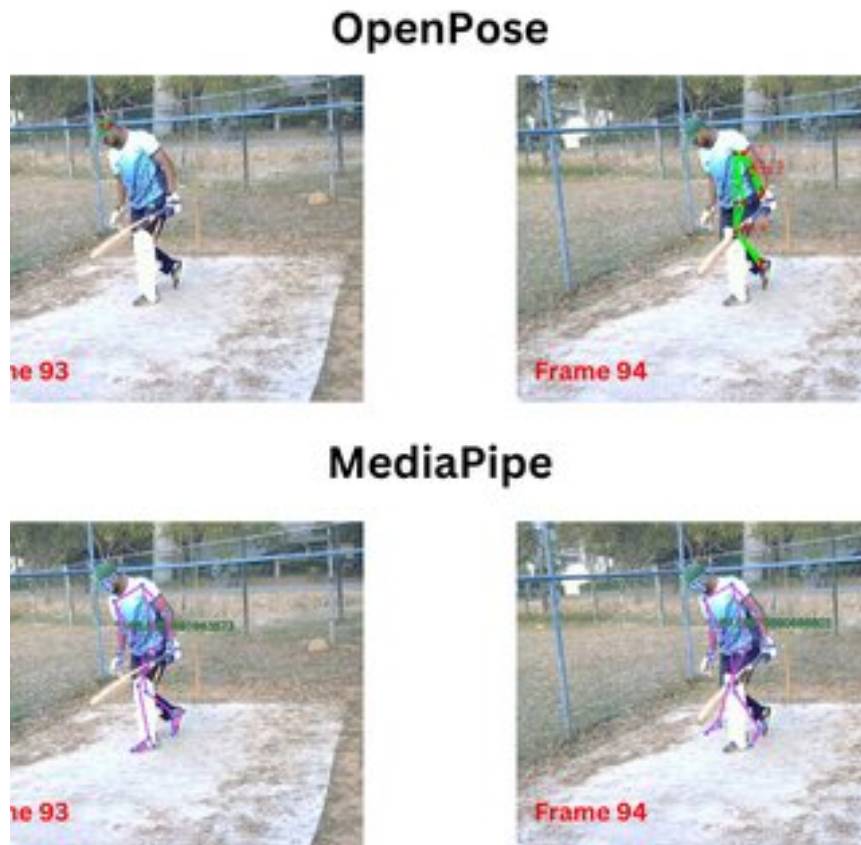


FIGURA 2.3: Captura de pose comparando OpenPose y Mediapipe.
Imagen adaptada de [9].

2.2.3. Triangulación 3D y Manejo de Oclusiones

Una vez obtenidas las coordenadas 2D del mismo punto clave en ambas cámaras, $p_A = (u_A, v_A)$ y $p_B = (u_B, v_B)$, el objetivo es recuperar la posición P_{3D} .

Geométricamente, P_{3D} se encuentra en la intersección de los dos rayos ópticos proyectados desde los centros de las cámaras y que pasan por p_A y p_B . Debido al ruido en la detección y calibración, estos rayos raramente se cortan perfectamente en el espacio (son rectas que se cruzan sin tocarse). Por tanto, el problema se formula como una optimización de mínimos cuadrados para encontrar el punto medio del segmento más corto que une ambos rayos [10].

Tal y como se ilustra en la Figura 2.4, la precisión de esta reconstrucción depende de la separación entre cámaras (línea base) y la resolución de los planos de imagen.

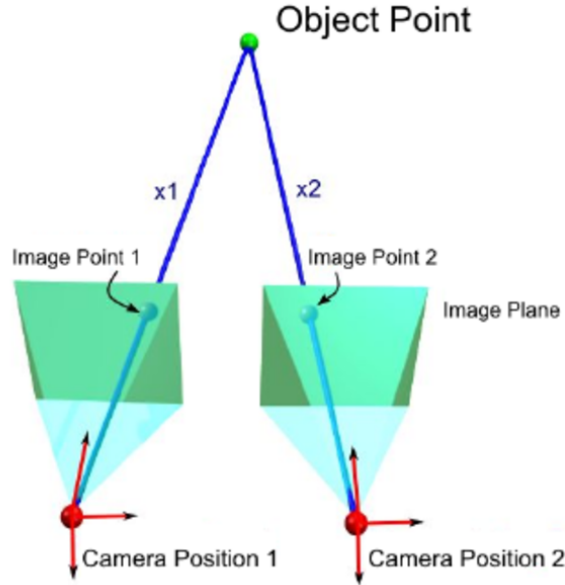


FIGURA 2.4: Principio geométrico de la triangulación estéreo. La posición del objeto en 3D se recupera mediante la intersección de los rayos proyectados desde los planos de imagen de dos cámaras calibradas. Imagen adaptada de [11].

Matemáticamente, si definimos las matrices de proyección de ambas cámaras como M_A y M_B , se puede plantear un sistema homogéneo lineal de la forma $AP_{3D} = 0$ (método DLT), donde la matriz A se compone de las filas de M_A y M_B ponderadas por las coordenadas de imagen detectadas:

$$AP_{3D} = 0 \quad \rightarrow \quad \min_{P_{3D}} ||AP_{3D}|| \quad (2.3)$$

El uso de dos vistas separadas espacialmente ofrece una ventaja crítica frente a una sola cámara RGB-D: la **resiliencia ante oclusiones**. Si el brazo del operario oculta su torso en la vista A, es probable que la vista B mantenga línea de visión directa. Esto permite al sistema mantener el seguimiento continuo de la demostración, descartando vistas ruidosas y aumentando la robustez del sistema LfD, cuya implementación detallada se describe en el Capítulo 3.

2.3. Detección de Objetos mediante Aprendizaje Profundo

Para que el robot pueda interactuar con las piezas de la tarea (muñecas, herramientas), es necesario localizar su posición en la imagen (u, v) de forma robusta frente a cambios de iluminación o fondos complejos. Mientras que la visión clásica se basa en características artesanales (color, bordes), este trabajo emplea el estado del arte en Aprendizaje Profundo (*Deep Learning*).

2.3.1. Redes Neuronales Convolucionales (CNNs)

Las CNNs son la columna vertebral de la visión artificial moderna. A diferencia de las redes densas tradicionales, las CNNs emplean la operación matemática de convolución para extraer características espaciales locales. Una capa convolucional aplica un conjunto de filtros (kernels) K sobre la imagen de entrada I , generando mapas de características que detectan patrones jerárquicos (desde bordes simples en las primeras capas hasta formas complejas en las últimas):

$$(I * K)(i, j) = \sum_m \sum_n I(i + m, j + n) K(m, n) \quad (2.4)$$

Estas arquitecturas suelen intercalar capas de *Pooling* para reducir la dimensionalidad y capas de activación no lineales (como ReLU o SiLU) para aprender funciones complejas [12].

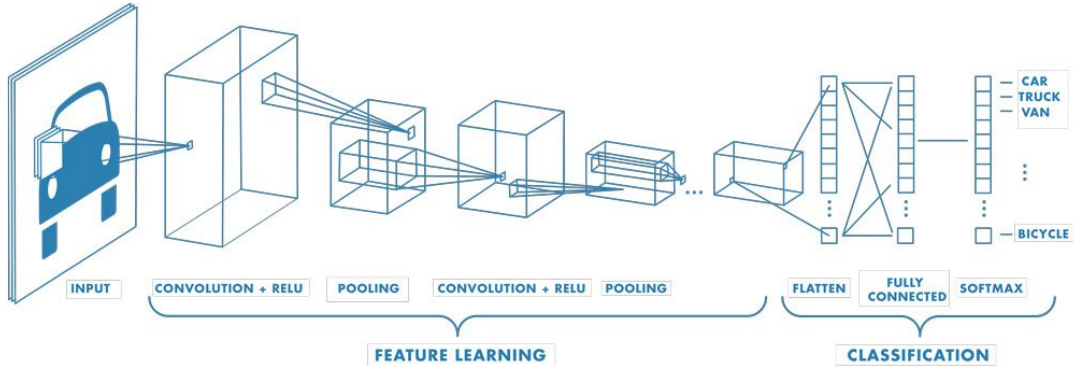


FIGURA 2.5: Ejemplo de red con múltiples capas convolucionales. Se aplican filtros a las imágenes de entrenamiento con distintas resoluciones, y la salida resultante de convolucionar cada imagen se emplea como entrada para la siguiente capa. Imagen extraída de [13].

2.3.2. Arquitectura YOLO (You Only Look Once)

Para la detección en tiempo real, existen dos paradigmas: detectores de dos etapas (como Faster R-CNN, precisos pero lentos) y detectores de una etapa.

En este TFG se utiliza “YOLO (You Only Look Once)”, específicamente la versión **YOLOv8**. La innovación disruptiva de YOLO [14] reside en reformular la detección de objetos no como un problema de clasificación de ventanas deslizantes, sino como un **único problema de regresión**.

La red divide la imagen de entrada en una cuadrícula de $S \times S$. Si el centro de un objeto cae en una celda, esa celda es responsable de detectarlo. Para cada celda, la red predice simultáneamente:

- Las coordenadas del *Bounding Box* (x, y, w, h).
- La confianza de que existe un objeto (P_{obj}).
- Las probabilidades de clase condicionales ($P(Clase_i | Objeto)$).

YOLOv8, desarrollado por Ultralytics [15], incorpora mejoras como la detección libre de anclas (*Anchor-Free*), lo que reduce la cantidad de hiperparámetros y mejora la

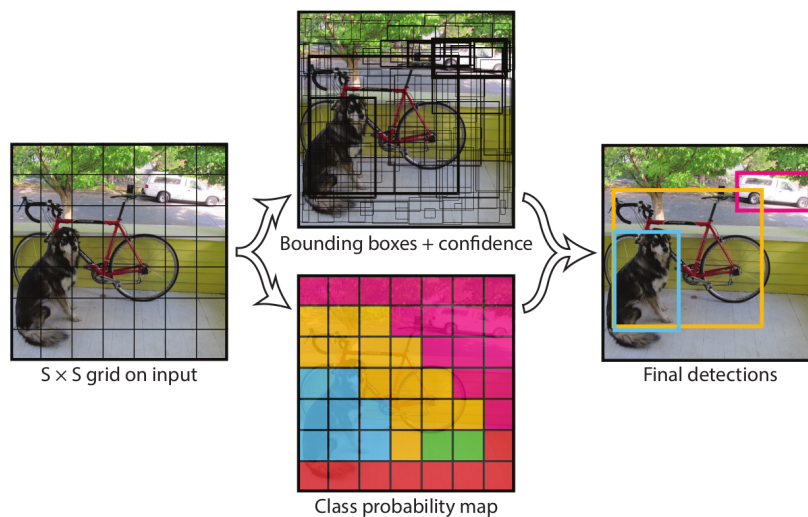


FIGURA 2.6: Arquitectura del modelo de detección YOLO. El sistema divide la imagen de entrada en una rejilla $S \times S$ y, para cada celda, predice simultáneamente las cajas delimitadoras (bounding boxes) y las probabilidades de clase. Imagen extraída de [14].

generalización en objetos de formas inusuales, ideal para las piezas manipuladas en este proyecto.

2.4. Aprendizaje por Demostración (LfD)

La programación tradicional de robots industriales requiere la definición explícita de cada punto de paso o la codificación de complejas leyes de control, lo cual es inviable para tareas dinámicas o para usuarios no expertos. El Aprendizaje por Demostración (LfD), también conocido como *Imitation Learning*, propone un cambio de paradigma: en lugar de programar el robot, se le “enseña” mediante ejemplos proporcionados por un maestro humano.

2.4.1. Taxonomía de las Estrategias de Enseñanza

Según la literatura clásica [1], existen tres modalidades principales para capturar la demostración:

- **Guiado Cinestésico (Kinesthetic Teaching):** El usuario mueve físicamente el brazo del robot (en modo “gravedad cero”) para realizar la tarea. Tiene la ventaja de eliminar el problema de la correspondencia (el robot ya está en su propio espacio articular), pero es difícil de ejecutar en tareas que requieren destreza fina o manipulación de objetos pequeños.
- **Teleoperación:** El usuario controla el robot a distancia mediante un joystick o dispositivo háptico.
- **Observación Pasiva (Passive Observation):** El robot observa al humano realizar la tarea con sus propias manos mediante sensores externos (visión). Este

es el enfoque más natural y el utilizado en este TFG. Sin embargo, introduce el desafío de traducir la cinemática humana a la del robot (*Correspondence Problem*).

Este problema, formalizado por Nehaniv y Dautenhahn [2], establece que transferir habilidades entre agentes con diferentes cuerpos (ej. humano y robot) requiere un mapeo no lineal entre sus espacios de estados y acciones.

2.4.2. Limitaciones de la Grabación Directa

Un enfoque ingenuo sería grabar la trayectoria articular $q(t)$ demostrada y reproducirla (*replay*). Sin embargo, esto carece de robustez: si la posición inicial del objeto cambia o si surge una perturbación externa, el robot fallará al seguir ciegamente el camino grabado. Por tanto, es necesario un modelo matemático que codifique la “intención” del movimiento y permita su generalización y adaptación espacial.

2.5. Primitivas de Movimiento Dinámico (DMPs)

Para dotar al sistema de capacidad de generalización, este trabajo utiliza las Primitivas de Movimiento Dinámico (DMPs), propuestas originalmente por Ijspeert et al. [16].

Las DMPs modelan una trayectoria compleja no como una función explícita del tiempo, sino como el comportamiento de un “sistema dinámico de segundo orden” (un sistema masa-resorte-amortiguador) que es perturbado por un término de fuerza no lineal aprendible. Esto garantiza propiedades críticas para la robótica colaborativa: estabilidad, convergencia a un objetivo y robustez ante perturbaciones.

2.5.1. Formulación Matemática

Una DMP se compone de dos subsistemas acoplados: el *Sistema Canónico* y el *Sistema de Transformación*.

El Sistema Canónico

Actúa como un reloj abstracto que decae monótonamente de 1 a 0. Se define mediante una ecuación diferencial de primer orden:

$$\tau \dot{s} = -\alpha_s s \quad (2.5)$$

Donde s es la variable de fase, α_s es una constante de tiempo y τ es un factor de escala temporal. Este sistema elimina la dependencia explícita del tiempo t , permitiendo que la ejecución del movimiento se pueda acelerar o ralentizar simplemente modificando τ .

El Sistema de Transformación

Es la ecuación que genera la aceleración deseada para el robot. Se inspira en un muelle amortiguado (PD control) atraído hacia un objetivo g (goal), al que se le suma un término de fuerza $f(s)$ para alterar la forma de la trayectoria:

$$\tau^2 \ddot{v} = K(g - y) - D(\tau v) + (g - y_0)f(s) \quad (2.6)$$

$$\tau \dot{y} = v \quad (2.7)$$

Donde:

- $y, v, \dot{v}$: Son la posición, velocidad y aceleración del sistema.
- g, y_0 : Son la posición objetivo y la posición inicial.
- K, D : Son las ganancias del resorte y el amortiguador. Para garantizar un comportamiento críticamente amortiguado (sin oscilaciones), se suele fijar $D = 2\sqrt{K}$.
- $f(s)$: Es el **Forcing Term** (término de fuerza), que contiene la “forma” aprendida del gesto humano.

2.5.2. Aprendizaje del Término de Fuerza

La “magia” de las DMPs reside en $f(s)$. Este término se modela como una suma ponderada de N funciones de base radial (Gaussianas) activadas por el sistema canónico s :

$$f(s) = \frac{\sum_{i=1}^N \psi_i(s) w_i}{\sum_{i=1}^N \psi_i(s)} s \quad (2.8)$$

Donde $\psi_i(s) = \exp(-h_i(s - c_i)^2)$ son las funciones Gaussianas centradas en c_i .

Durante la fase de entrenamiento (demostración), dada una trayectoria humana $(y_{demo}, \dot{y}_{demo}, \ddot{y}_{demo})$, podemos despejar f_{target} de la ecuación dinámica encontrar los pesos w_i mediante regresión lineal localmente ponderada (LWR). Esto permite que el robot “copie” cualquier forma de movimiento suave (una curva, una letra, un gesto de agarre) y luego la reproduzca adaptándose a nuevos puntos de inicio y_0 y final g .

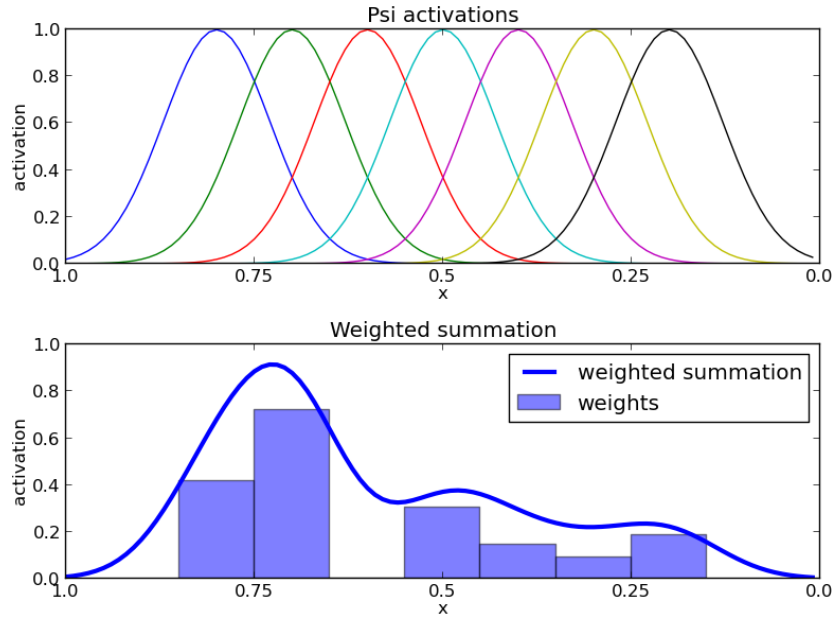


FIGURA 2.7: Funcionamiento del término de fuerza. Arriba: Activación secuencial de las funciones de base Gaussianas (ψ_i). Abajo: Reconstrucción de la forma de la trayectoria mediante la suma ponderada. Imagen adaptada de [17].

2.6. Análisis Comparativo con Estrategias Existentes

Para contextualizar la solución propuesta, es pertinente compararla con otras aproximaciones recientes en la literatura de LfD para manipulación.

2.6.1. End-to-End vs. Pipeline Modular

Existen enfoques basados en aprendizaje “End-to-End” (de extremo a extremo), donde una única red neuronal profunda recibe los píxeles de la imagen y emite directamente los comandos de velocidad del robot (visuomotor policies), como los propuestos por el equipo de Google Robotics con RT-1 o RT-2 [18]. Aunque potentes, estos sistemas requieren miles de horas de entrenamiento y GPUs de clúster. En contraste, la arquitectura modular propuesta en este TFG (Visión $\rightarrow$ Estado $\rightarrow$ DMP $\rightarrow$ Control) permite una trazabilidad del error mucho mayor: si el robot falla, podemos saber si fue fallo de detección (YOLO) o de ejecución (DMP), algo imposible en las “cajas negras” del End-to-End. Además, permite el despliegue en hardware modesto (PC estándar con ROS).

2.6.2. Sistemas con Marcadores vs. Markerless

Sistemas clásicos de captura como Vicon [19] o OptiTrack [20] ofrecen precisión sub-milimétrica pero requieren que el operario vista trajes con marcadores reflectantes, lo cual es invasivo y poco práctico en una línea de producción real. El enfoque *markerless* (sin marcadores) adoptado en este trabajo mediante MediaPipe y Triangulación

Estéreo, aunque sacrifica cierta precisión absoluta, gana en usabilidad y reduce drásticamente el coste de implementación, democratizando el acceso a la tecnología LfD para PYMES.

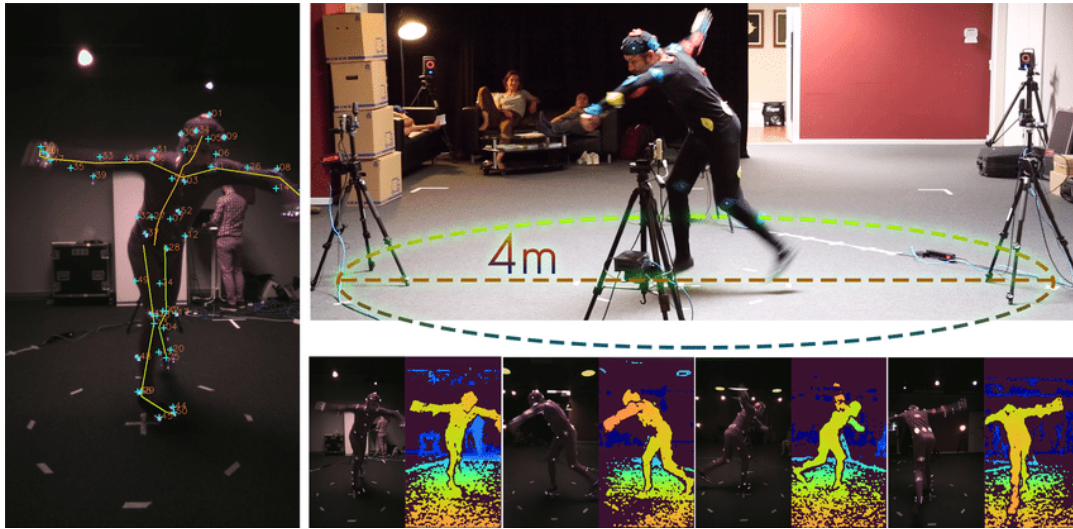


FIGURA 2.8: Ejemplo del sistema de captura marker-based, para ilustrar la cantidad de elementos usados en este tipo de sistemas. Imagen adaptada de [21].

3 Sistemas de percepción

El éxito de un sistema de Aprendizaje por Demostración (LfD) pasivo reside en la capacidad de capturar la cinemática del experto humano con alta fidelidad y sin marcadores invasivos. Este capítulo detalla el desarrollo del módulo de visión artificial, diseñado para reconstruir la escena 3D a partir de un sistema estéreo.

3.1. Arquitectura del Sistema de Percepción

El sistema de percepción se ha diseñado bajo un enfoque modular, separando la adquisición de imágenes, el procesamiento neuronal y la reconstrucción geométrica.

3.1.1. Hardware y Disposición de Sensores

Se ha optado por una configuración de visión estéreo compuesta por dos cámaras Intel RealSense D415i [6]. Aunque estas cámaras poseen sensores de profundidad activos, en este trabajo se utilizan principalmente sus flujos RGB sincronizados para realizar una triangulación estéreo personalizada.

La disposición de dos puntos de vista angulados permite mitigar el problema de las oclusiones (auto-occlusiones del brazo sobre el cuerpo o sobre el objeto manipulado), asegurando que al menos una cámara tenga línea de visión directa con los puntos de interés durante la manipulación.

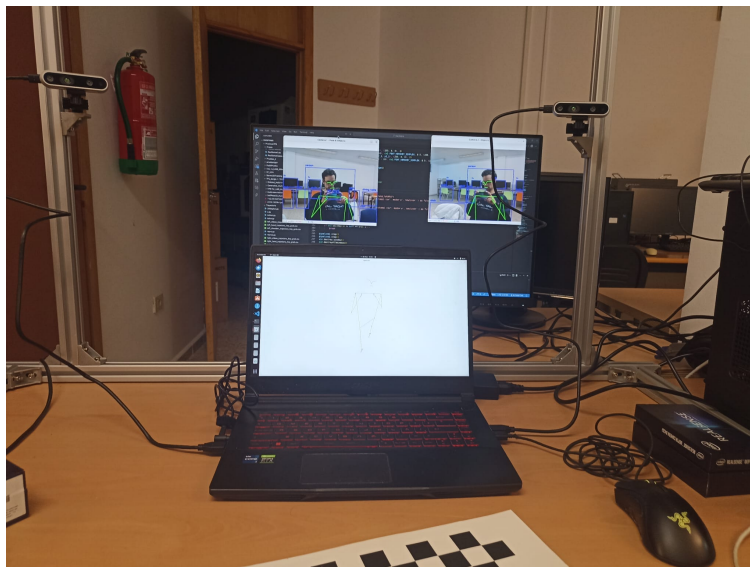


FIGURA 3.1: Setup inicial de las primeras pruebas en laboratorio. Se observa el sistema YOLO detectando objetos y MediaPipe detectando la pose del operario.

3.1.2. Stack Tecnológico y Flujo de Datos

El pipeline de software integra varias librerías del estado del arte, seleccionadas por su eficiencia en sub-tareas específicas:

- **pyrealsense2:** Interfaz de bajo nivel para la adquisición sincronizada de frames. Propia para usar las cámaras Intel RealSense en Python.
- **MediaPipe:** Framework de Google utilizado para la extracción del esqueleto humano y los landmarks de las manos con baja latencia.
- **YOLOv8 (Ultralytics):** Red neuronal convolucional para la detección de objetos, ejecutada en formato ONNX para optimizar la inferencia, y re-entrenada para detectar la muñeca de juguete usada para las demostraciones.
- **OpenCV & NumPy:** Motor matemático para la manipulación de matrices, calibración y cálculos de geometría epipolar.
- **Open3D:** Herramienta de visualización para la validación de los puntos reconstruidos en tiempo real.

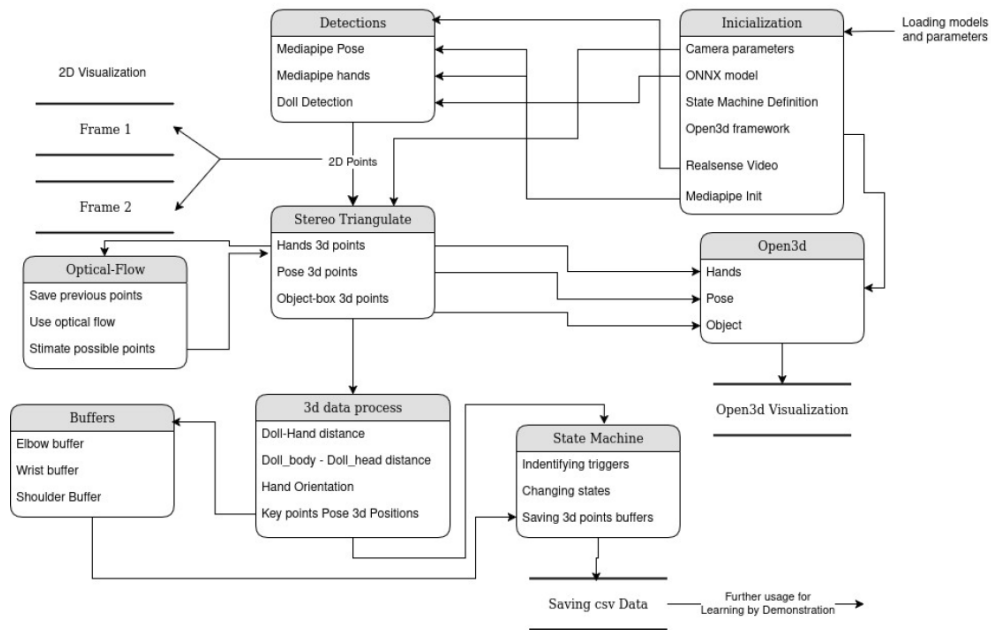


FIGURA 3.2: Diagrama de flujo del sistema de percepción. El sistema procesa paralelamente las detecciones 2D para posteriormente triangular la posición 3D y alimentar la máquina de estados.

3.2. Calibración del Sistema Estéreo

Para poder reconstruir información 3D a partir de dos planos de imagen 2D, es imperativo conocer la relación geométrica precisa (rotación R y traslación T) entre las dos cámaras.

El proceso de calibración se llevó a cabo utilizando el algoritmo de Zhang [7] implementado en `cv2.StereoCalibrate`. Se capturó un dataset de 20 pares de imágenes

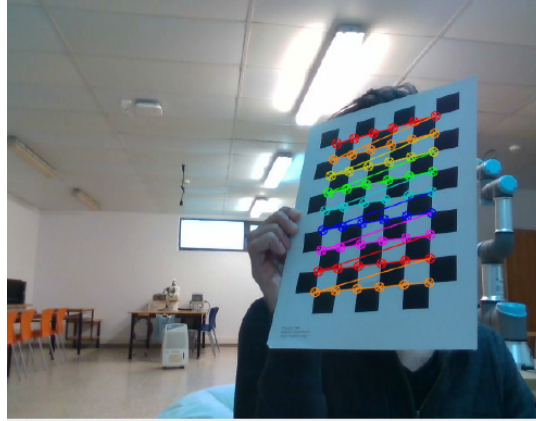


FIGURA 3.3: Captura de pantalla del proceso de detección de esquinas en el patrón de ajedrez para la calibración.

sincronizadas de un patrón de ajedrez (*chessboard*) en diferentes orientaciones y profundidades. El resultado de este proceso son las matrices intrínsecas (K_1, K_2) y la matriz de transformación extrínseca ${}^{C1}T_{C2}$, que permite transformar cualquier punto del sistema de coordenadas de la Cámara 2 a la Cámara 1.

3.2.1. Calibración Extrínseca Cámara-Robot

Una vez calibrado el par estéreo, el sistema de visión proporciona coordenadas 3D (P_{cam}) referidas al centro óptico de la cámara principal. Para que el robot pueda utilizar esta información, es necesario transformar estos puntos a su propio sistema de coordenadas base (P_{base}).

Esto requiere determinar con precisión la matriz de transformación homogénea ${}^{Base}T_{Cam}$. Para ello, se realizó un procedimiento de calibración extrínseca “Robot-Mundo” (a veces denominado *hand-eye* inverso).

Como se muestra en la Figura 3.4, se acopló rígidamente el patrón de calibración al efector final del robot UR3e. Se programó al robot para mover el patrón a una serie de N poses conocidas (T_{TCP}^{Base}) dentro del volumen de trabajo. En cada pose, el sistema de visión detectó la posición del patrón respecto a la cámara ($T_{Pattern}^{Cam}$).

Resolviendo el problema de optimización asociado ($AX = XB$), se obtuvo la matriz ${}^{Base}T_{Cam}$ que permite mapear cualquier detección visual directamente al espacio de trabajo del manipulador. De manera análoga a la calibración estéreo, este proceso minimiza el error de reproyección para asegurar la consistencia geométrica entre ambos sistemas de coordenadas.



FIGURA 3.4: Configuración experimental para la calibración extrínseca cámara-base. El robot UR3e posiciona con precisión el patrón de ajedrez en el campo de visión de las cámaras RealSense. Este proceso permite obtener la matriz de transformación que relaciona el sistema de coordenadas de visión con la base del robot.

3.3. Detección y Reconocimiento en 2D

El procesamiento de cada imagen se divide en dos ramas: seguimiento humano y detección de objetos.

3.3.1. Estimación de Pose Humana (MediaPipe)

Se emplea el modelo MediaPipe Pose y MediaPipe Hands para extraer las coordenadas pixel (u, v) de las articulaciones clave (hombro, codo, muñeca) y los dedos. A diferencia de métodos “bottom-up” como OpenPose, MediaPipe utiliza un enfoque “top-down” con seguimiento, lo que garantiza una mayor estabilidad temporal y reduce el *jitter* (temblor) en la señal de salida, algo crítico para no introducir ruido en el aprendizaje del robot (ver Figura 3.7).

3.3.2. Detección de Objetos (YOLOv8 Personalizado)

Para la detección de las piezas de la tarea (cabeza y cuerpo de la muñeca), los modelos pre-entrenados genéricos (COCO dataset) no eran suficientes. Se desarrolló un detector específico siguiendo estos pasos:

1. **Creación del Dataset:** Se capturaron imágenes del entorno real y se etiquetaron manualmente las clases `doll_head` y `doll_body` utilizando la herramienta CVAT (*Computer Vision Annotation Tool*).

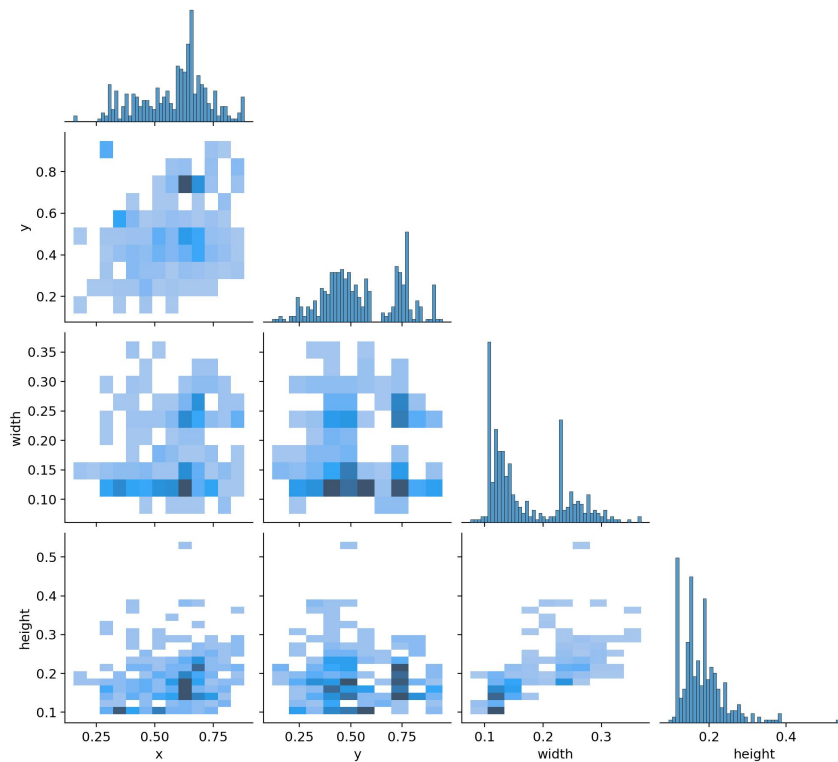


FIGURA 3.5: Correlograma de etiquetas del dataset de entrenamiento. Se muestra la distribución espacial (x, y) y dimensional $(width, height)$ de las instancias anotadas. La concentración de datos en el eje $y > 0,5$ refleja la disposición física del experimento, con los objetos situados sobre la mesa de trabajo en la mitad inferior del encuadre.

El análisis de la distribución de etiquetas (Figura 3.5) permite verificar la consistencia del dataset generado. Los histogramas en la diagonal confirman que las instancias de `doll_head` y `doll_body` mantienen dimensiones relativas coherentes a lo largo de las muestras. Asimismo, la densidad en los mapas de calor evidencia que la mayor parte de las interacciones ocurren en la zona central-inferior del espacio de trabajo, lo que valida que el dataset cubre correctamente la región de interés (ROI) donde opera el robot.

2. **Fine-tuning:** Se re-entrenó un modelo YOLOv8 Nano (`yolov8n`) sobre este dataset personalizado. Se eligió la versión “Nano” para priorizar el bajo peso del modelo y la capacidad de generalización en hardware no especializado.
3. **Exportación a ONNX:** Para desacoplar el sistema de la librería pesada de PyTorch en tiempo de ejecución, el modelo entrenado se exportó al formato abierto ONNX (*Open Neural Network Exchange*) [22], ejecutándose mediante el módulo `cv2.dnn` de OpenCV [23].



FIGURA 3.6: Muestra del conjunto de validación con las etiquetas de verdad terrena (*ground truth*) utilizadas para el reentrenamiento de YOLOv8n.

3.4. Reconstrucción 3D y Lógica de Escena

3.4.1. Triangulación Estéreo

Una vez obtenidos los puntos 2D correspondientes en ambas cámaras (p_{left}, p_{right}), se aplica el principio de triangulación lineal. Dados los parámetros de calibración, se resuelven las ecuaciones de proyección para encontrar el punto $P(x, y, z)$ en el espacio 3D donde intersectan los rayos ópticos. Este proceso genera un “esqueleto 3D” del operario y los centroides de los objetos en tiempo real (ver Figura 3.7).

3.4.2. Máquina de Estados Basada en Eventos

Para traducir el flujo continuo de datos geométricos en una secuencia de acciones discretas aprendibles (segmentación semántica de la tarea), se implementó una Máquina de Estados Finitos (FSM).

Como se detalla en la Tabla 3.1, las transiciones entre estados se disparan mediante *triggers* geométricos, principalmente basados en la distancia euclídea entre la mano del operario y el objeto.

Finalmente, los datos de las articulaciones (hombro, codo, muñeca) segmentados por esta máquina de estados se almacenan en archivos CSV formateados para alimentar el algoritmo de Primitivas de Movimiento Dinámico (DMPs) descrito en el capítulo siguiente.

CUADRO 3.1: Definición de la Máquina de Estados para la segmentación de la tarea.

Estado	Descripción	Condición de Transición (Trigger)
Idle	Robot en espera.	Dist Mano-Objeto < Umbral A.
Approaching	Aproximación a la pieza.	Dist Mano-Objeto < Umbral de Agarre.
Grasping	Interacción física/agarre.	Cambio significativo en ángulo de muñeca.
Rotating/Pulling	Manipulación/desmolde.	Dist Cabeza-Cuerpo > Umbral de Separación.
Final	Tarea completada.	-

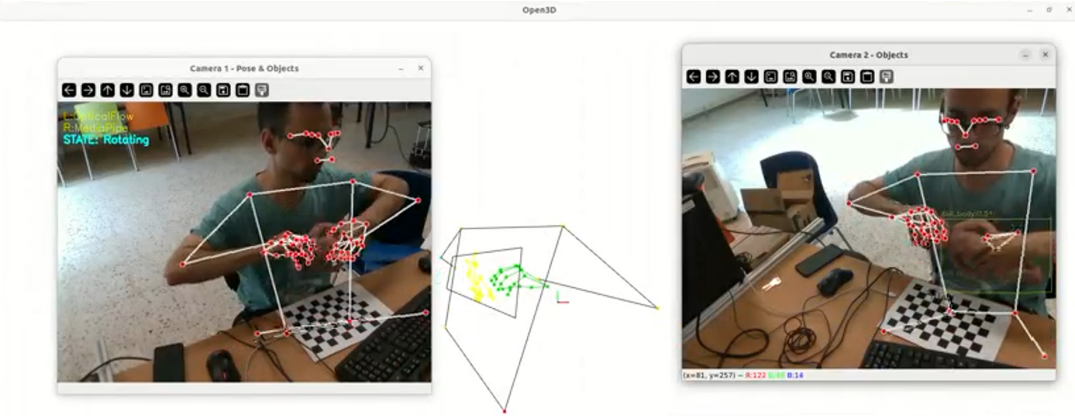


FIGURA 3.7: Visualización en tiempo real del sistema de percepción completo. A los lados: Vistas de cámara 1 y 2 con detección de pose (MediaPipe) y objetos (YOLO). Centro: Reconstrucción 3D del esqueleto mediante triangulación estéreo en Open3D. Nótese el indicador de estado actual (“Rotating”) en la vista izquierda, resultado de la máquina de estados.

4 Control

Una vez digitalizado el movimiento humano, el siguiente desafío es transferirlo al robot. Este proceso no es trivial debido a las profundas diferencias cinemáticas y ambientales entre el demostrador y el manipulador UR3e. Este capítulo detalla los problemas encontrados con el control cartesiano estándar y la solución geométrica desarrollada para superarlos.

4.1. El Desafío de la Correspondencia Morfológica

Inicialmente, se planteó un control basado en el seguimiento de pose cartesiana (enviar al robot la posición (x, y, z) de la muñeca humana). Sin embargo, durante las pruebas preliminares se identificaron dos limitaciones críticas:

1. **Singularidades y Soluciones Múltiples:** Los solvers de cinemática inversa numéricos a menudo convergían a soluciones de “codo abajo” (*elbow-down*), naturales para un humano de pie, pero catastróficas para un brazo robótico anclado a una mesa, ya que provocaban colisiones inmediatas con la superficie de trabajo.
2. **Restricciones del Espacio de Trabajo:** El operario humano tiene el hombro libre y el codo colgando hacia el suelo. El UR3e, por el contrario, tiene su base fija sobre la mesa y sus articulaciones limitadas. Intentar copiar literalmente la coordenada Z del codo humano resultaba en comandos inalcanzables.

Para solventar esto, se descartó el uso de cinemática inversa genérica en favor de un “Mapeo Geométrico Directo en el Espacio Articular”, integrando una técnica de reflexión morfológica.

4.2. Algoritmo de Mapeo Cinemático

El algoritmo desarrollado procesa los **keyframes** capturados por el sistema de visión y genera una configuración articular q_{robot} segura y coherente. El proceso consta de tres etapas, ilustradas en el script de validación desarrollado (`Validate and compare kinematic mapping` [24]).

4.2.1. Transformación y Reflexión Morfológica

En primer lugar, los puntos clave se transforman del sistema de coordenadas de la cámara al de la base del robot mediante la matriz de calibración ${}^{Base}T_{Cam}$.

Para resolver el problema de la colisión con la mesa, se aplica una “reflexión morfológica” sobre el plano horizontal que pasa por el hombro. Matemáticamente, si

definimos el vector brazo humano como $\vec{v}_{se} = P_{codo} - P_{hombro}$, la posición virtual del codo del robot P'_{codo} se calcula invirtiendo la componente vertical:

$$P'_{codo_z} = P_{hombro_z} - (P_{codo_z} - P_{hombro_z}) \quad (4.1)$$

Esta operación *espejo* garantiza que, cuando el humano baja el brazo (movimiento natural), el robot sube el codo (configuración “elbow-up”), manteniendo la misma extensión funcional pero evitando la colisión con la mesa.

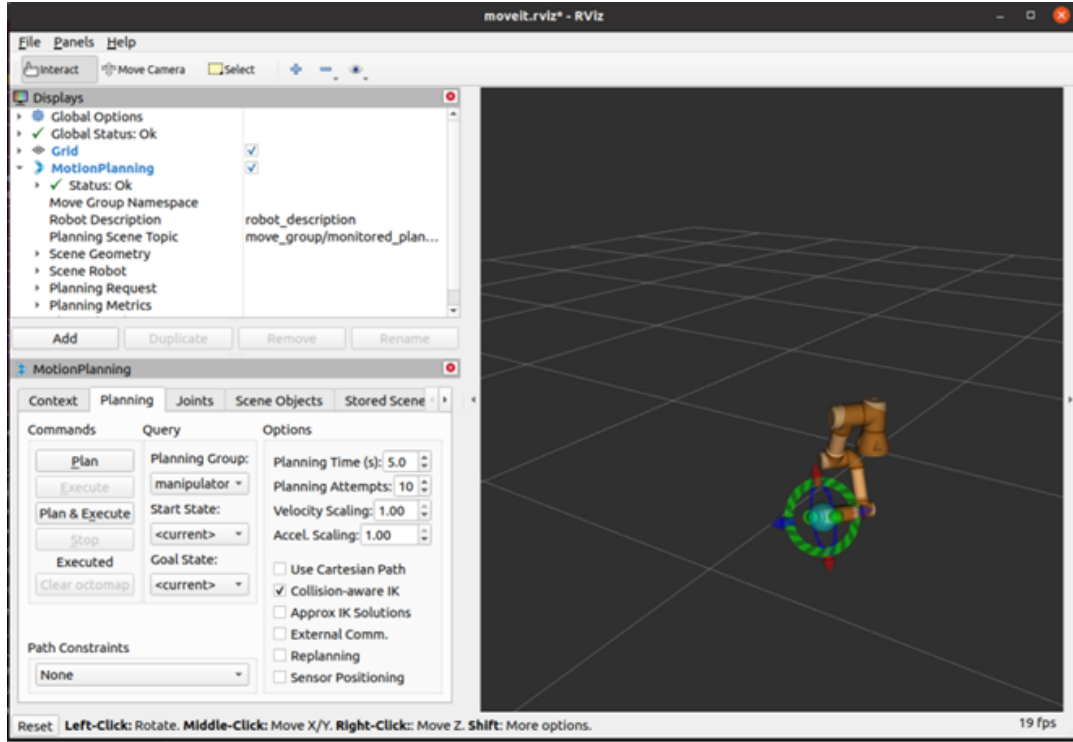


FIGURA 4.1: Entorno de simulación en RViz, se ilustra que si aplicamos el DMP sin hacer la reflexion morfológica, quedaría debajo del suelo en simulacion, lo que corresponde a la mesa de apoyo.

4.2.2. Extracción de Ángulos Relativos

En lugar de mapear posiciones absolutas, se extraen los ángulos relativos intrínsecos de la pose humana utilizando álgebra vectorial. Se define un sistema de coordenadas dinámico en el torso del operario (formado por los hombros y la cadera) para hacer la medición robusta a la orientación del usuario.

A partir de este marco de referencia, se calculan tres ángulos principales:

- **Hombro Abducción/Aducción (θ_{abd}):** Ángulo del brazo respecto al plano frontal.
- **Hombro Flexión/Extensión (θ_{flex}):** Ángulo de elevación del brazo.
- **Codo (θ_{elbow}):** Ángulo entre el vector del húmero y el antebrazo.

Estos ángulos se obtienen mediante el producto escalar de los vectores normalizados proyectados en los planos anatómicos correspondientes.

4.2.3. Mapeo Lineal al Espacio Articular del Robot

Finalmente, los ángulos humanos se traducen a las articulaciones del UR3e mediante una ley de control lineal que incluye factores de escala S y offsets de calibración O , determinados experimentalmente mediante la comparación de una “Pose en T” de referencia:

$$q_{robot}^i = O_{robot}^i + S^i \cdot (\theta_{human}^i - O_{human}^i) \quad (4.2)$$

Donde S^i suele ser $-1,0$ para invertir el sentido de giro cuando la morfología del robot es opuesta a la humana. Este enfoque directo evita el cálculo de la cinemática inversa, eliminando el riesgo de singularidades y garantizando un comportamiento predecible.

4.3. Validación y Simulación Cinemática

Dada la peligrosidad de probar algoritmos de control directamente sobre hardware real, se desarrolló un entorno de validación en Python previo al despliegue.

Este software carga las demostraciones grabadas y simula el comportamiento del robot aplicando las ecuaciones de “Cinemática Directa” (*Forward Kinematics*) del UR3e utilizando sus parámetros de Denavit-Hartenberg estándar [4].

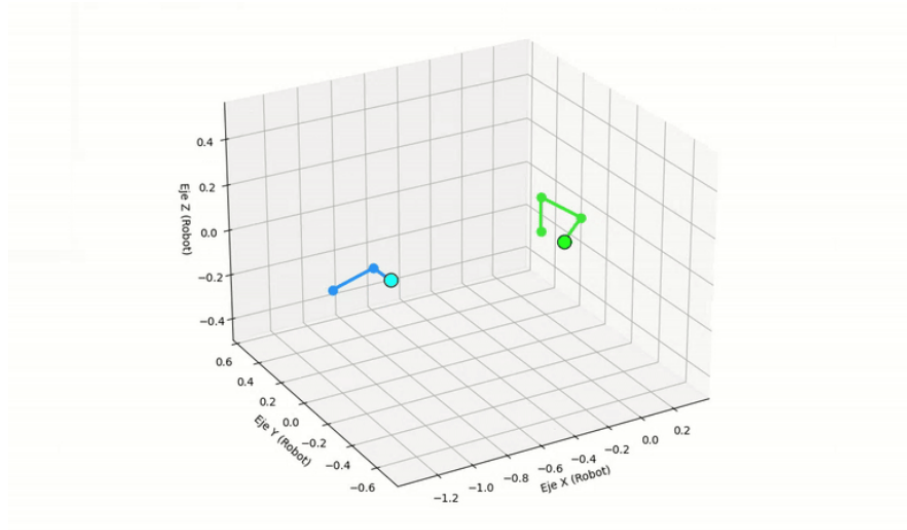


FIGURA 4.2: Visualización del entorno de validación desarrollado. En azul: esqueleto humano capturado y transformado. En verde: reconstrucción cinemática del robot UR3e ejecutando los ángulos calculados. Nótese la configuración “codo arriba” del robot frente a la “codo abajo” del humano, resultado de la reflexión morfológica.

La herramienta genera una visualización 3D comparativa (Figura 4.2) que permite verificar frame a frame si la reflexión morfológica y el mapeo angular producen movimientos lógicos y seguros antes de enviar cualquier comando al controlador físico del robot.

4.4. Pipeline de Aprendizaje de Primitivas (DMP Learning)

Una vez validada la coherencia del mapeo cinemático frame a frame, el siguiente paso es dotar al robot de la capacidad de aprender y generalizar la tarea completa. Para ello, se ha implementado un pipeline de entrenamiento offline (script `learn_dmp.py`) que procesa el conjunto de demostraciones capturadas y genera los modelos dinámicos necesarios.

4.4.1. Preprocesamiento y Normalización Temporal

Las demostraciones humanas son intrínsecamente variables: cada ejecución tiene una duración distinta (T_i) y contiene ruido de alta frecuencia proveniente del sistema de visión. Para poder procesarlas conjuntamente, es necesario normalizarlas.

El sistema carga los archivos CSV segmentados por la máquina de estados y aplica las siguientes etapas de acondicionamiento de señal:

1. **Filtrado de Ruido:** Se aplica un filtro de Savitzky-Golay (ventana $w = 11$, orden polinomial $k = 3$) a las trayectorias articulares generadas. Este filtro es ideal para preservar los picos y formas de la señal mientras elimina el ruido gaussiano del sensor [25].
2. **Re-muestreo (Resampling):** Dado que las DMPs discretas requieren vectores de longitud fija, todas las trayectorias se re-muestran a un número constante de puntos ($N = 200$) mediante interpolación lineal. Esto alinea temporalmente todas las demostraciones en un eje de fase común $s \in [0, 1]$.

4.4.2. Extracción de la Trayectoria Maestra (Averaging)

En lugar de entrenar el robot con una única demostración (que podría contener errores o movimientos espurios), este trabajo propone un enfoque estadístico. Se calcula una “Trayectoria Maestra” (μ_{traj}) promediando las demostraciones válidas en el espacio articular del robot:

$$\mu_{traj}[n] = \frac{1}{M} \sum_{i=1}^M \tau_i[n] \quad \forall n \in [0, N] \quad (4.3)$$

Donde M es el número de demostraciones (ej. 10 repeticiones) y τ_i es la trayectoria i -ésima ya mapeada al espacio del robot. Este promedio actúa como un potente filtro paso bajo espacial, extrayendo la “intención media” del operario y descartando las variaciones individuales no deseadas.

4.4.3. Entrenamiento y Serialización del Modelo

Finalmente, la Trayectoria Maestra se utiliza para entrenar el modelo de Primitivas de Movimiento Dinámico. Se ha utilizado la librería `pydmps` configurada con $N_{bfs} = 30$ funciones de base gaussianas, lo que ofrece un compromiso óptimo entre la capacidad de modelado de curvas complejas y la suavidad del movimiento resultante.

El algoritmo de aprendizaje (Locally Weighted Regression - LWR) [26] calcula los pesos w_i que minimizan el error entre la aceleración generada por la DMP y la aceleración de la trayectoria maestra. El resultado es un conjunto de archivos serializados (`.pkl`) para cada fase de la tarea (ej. `dmp_grasping.pkl`, `dmp_rotating.pkl`), que contienen la “receta dinámica” del movimiento, lista para ser cargada por el controlador en tiempo real descrito en el siguiente capítulo.

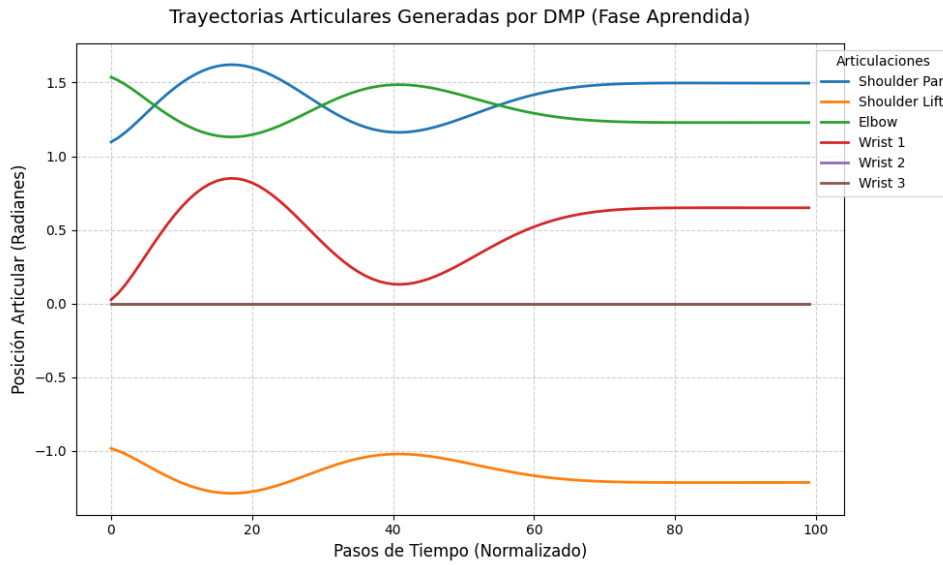


FIGURA 4.3: Trayectorias articulares generadas por el modelo DMP para la fase de aproximación. Se observa la suavidad de las curvas (C^2 continuo) y la convergencia asintótica al objetivo, eliminando el ruido de alta frecuencia presente en la captura humana original. Cada color representa el perfil de posición de una articulación del robot (q_1 a q_6).

5 Experimentación

Tras el entrenamiento de los modelos dinámicos (DMPs) y la validación cinemática en simulación, se procedió a la integración final en el entorno real. Este capítulo describe la arquitectura de despliegue utilizada, la estrategia de control implementada para garantizar la seguridad del manipulador UR3e y los resultados obtenidos, tanto cuantitativos como cualitativos, en la ejecución de la tarea de desmolde.

5.1. Arquitectura de Despliegue

El sistema se ha validado en un entorno de laboratorio controlado. La arquitectura de hardware y software se compone de los siguientes elementos:

- **Manipulador:** Universal Robots UR3e con gripper de dos dedos conectado a las E/S digitales de herramienta.
- **Controlador:** PC con Ubuntu 20.04 y ROS Noetic, ejecutando el driver oficial `ur_robot_driver` para la comunicación de bajo nivel.
- **Planificador:** Se utiliza **MoveIt!** como interfaz de alto nivel para la gestión de cinemática inversa y prevención de autocolisiones.

5.2. Estrategia de Control y Ejecución

Para ejecutar las trayectorias aprendidas en el robot físico, se desarrolló un nodo de orquestación en Python (`ur3e_dmp_player.py`). Este nodo no se limita a reproducir ciegamente las salidas del modelo DMP, sino que implementa una capa de seguridad y una estrategia de control híbrida.

5.2.1. Capa de Seguridad y Límites

Dado que los modelos generativos pueden, en ocasiones, producir valores fuera de los rangos físicos del robot (especialmente en las articulaciones de la muñeca), se implementó un filtro de saturación dinámico:

$$q_{cmd}^i = \text{clip}(q_{dmp}^i, q_{min}^i, q_{max}^i) \quad (5.1)$$

Adicionalmente, se aplicó un factor de escalado de velocidad ($v_{scale} = 0,5$) en el controlador de trayectoria de MoveIt para permitir la supervisión humana y la parada de emergencia durante las pruebas iniciales.

5.2.2. Control Híbrido para la Fase de Manipulación

Uno de los hallazgos experimentales más relevantes fue el comportamiento durante la fase de rotación (*grasping_to_rotating*). Al intentar mapear la rotación de la muñeca humana directamente a todo el brazo robótico, se observaron movimientos parásitos de gran amplitud en el codo y el hombro para compensar pequeños cambios de orientación, un fenómeno conocido como *movimiento nulo* en robots redundantes.

Para solucionar esto, se implementó una estrategia de “Desacople Cinemático” en el script de control:

1. Durante las fases de aproximación (*Initial-Idle*), el robot sigue la trayectoria completa de 6 grados de libertad generada por la DMP.
2. Al entrar en la fase de rotación, el controlador bloquea las primeras 5 articulaciones ($q_1 \dots q_5$) en su última posición conocida.
3. La señal de control de la DMP se redirige exclusivamente a la última articulación (q_6 o *wrist_3*), permitiendo una rotación pura de la herramienta sin alterar la pose global del brazo.

Esta estrategia híbrida resultó crítica para lograr un desmolde exitoso sin colisiones con la mesa de trabajo.

5.3. Evaluación del Subsistema de Percepción

En esta sección se analiza cuantitativamente el desempeño del módulo de visión, un aspecto crítico para garantizar la fiabilidad del sistema LfD.

5.3.1. Métricas de Entrenamiento (YOLOv8)

Para validar la robustez del detector de objetos, se analizaron las curvas de aprendizaje generadas durante el proceso de *fine-tuning* (Figura 5.1).

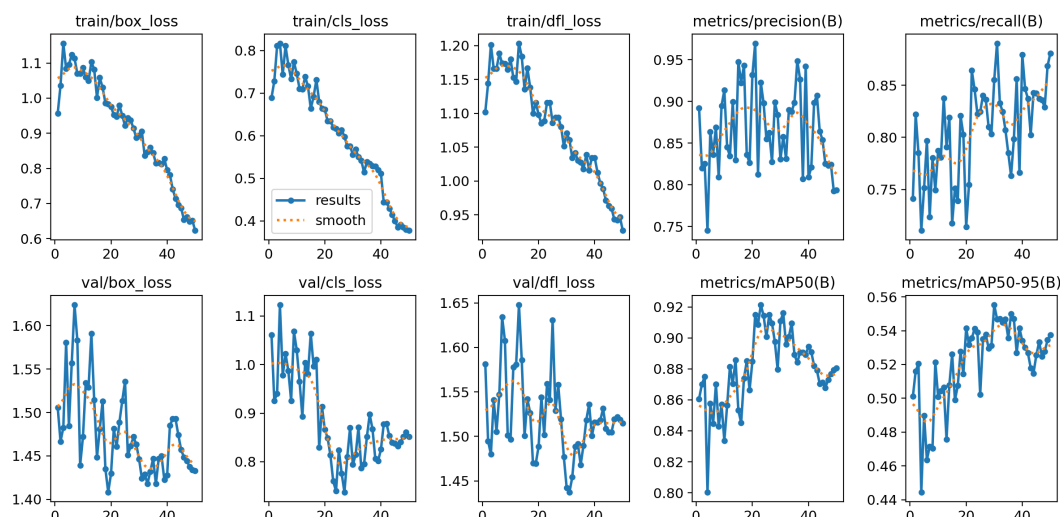


FIGURA 5.1: Evolución de las funciones de pérdida y métricas de desempeño durante el entrenamiento del detector. Fila superior: Pérdidas en conjunto de entrenamiento y métricas globales. Fila inferior: Pérdidas en conjunto de validación y métricas mAP. Se observa una convergencia estable a partir de la época 40.

El análisis de estas gráficas permite extraer las siguientes conclusiones:

- **Minimización del Error (Losses):** Se observa una tendencia descendente clara en las funciones de pérdida de caja (*box_loss*), clasificación (*cls_loss*) y focal de distribución (*df_l_loss*) tanto en entrenamiento como en validación. Esto indica que la red neuronal ha aprendido progresivamente a ajustar las coordenadas de las cajas delimitadoras y a distinguir entre la cabeza y el cuerpo de la muñeca sin sufrir de sobreajuste (*overfitting*) severo.
- **Precisión y Exhaustividad (Recall):** Las métricas de *Precision* y *Recall* (esquina superior derecha) muestran oscilaciones típicas de los algoritmos estocásticos, pero con una tendencia ascendente, estabilizándose por encima del 85 % y 80 % respectivamente en las últimas épocas.
- **Mean Average Precision (mAP):** La métrica más representativa, el mAP@50 (precisión media con un umbral de IoU del 50 %), alcanza valores cercanos a **0.92** en la validación. El mAP@50-95 (una métrica más estricta que promedia varios umbrales) alcanza un **0.54**. Estos valores confirman que el modelo es altamente fiable para la tarea, garantizando detecciones precisas incluso bajo condiciones de oclusión parcial.

Complementariamente, la matriz de confusión (Figura 5.2) revela el comportamiento por clases:

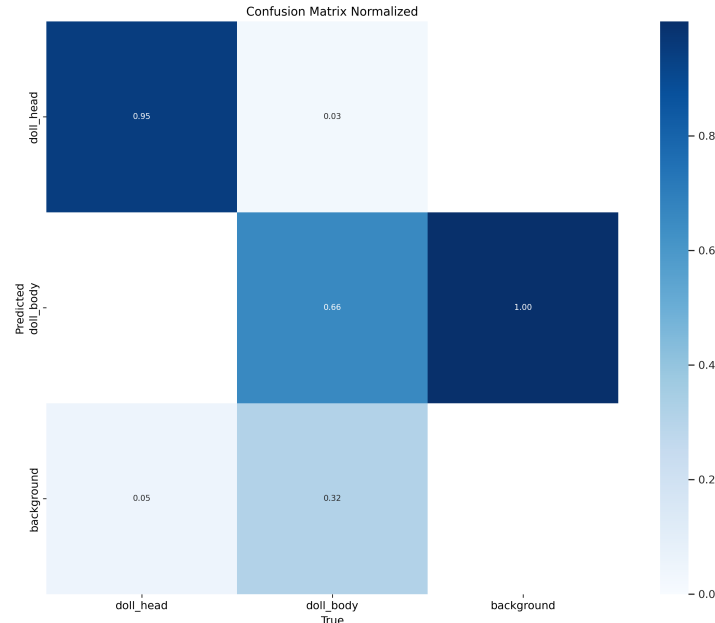


FIGURA 5.2: Matriz de confusión normalizada del modelo YOLOv8n re-entrenado. Se observa una alta precisión en la clase crítica `doll_head` (95 %), mientras que la clase `doll_body` presenta una mayor tasa de falsos negativos debido a oclusiones parciales durante la manipulación.

5.3.2. Análisis de Latencia y Tiempo de Cómputo

Además de la precisión, se evaluó el rendimiento temporal del pipeline de visión, factor limitante para la interacción en tiempo real. En el hardware utilizado (PC estándar sin GPU dedicada de alto rendimiento), el sistema alcanzó una tasa de procesamiento promedio estable.

El desglose aproximado del tiempo de cómputo por frame es:

- **Adquisición y Sincronización:** ≈ 10 ms.
- **Inferencia MediaPipe (CPU):** ≈ 30 ms.
- **Inferencia YOLOv8n (ONNX):** ≈ 15 ms.
- **Triangulación y Lógica:** < 5 ms.

Esto resulta en una latencia total aproximada de 60 ms (≈ 16 FPS), lo cual se considera aceptable para capturar gestos humanos a velocidad natural, permitiendo una experiencia de demostración fluida.

5.4. Resultados de la Ejecución Robótica

Para la validación final del control, se realizaron ejecuciones de las DMPs aprendidas utilizando los datos capturados por el sistema de percepción evaluado anteriormente. La validación consistió en 10 ejecuciones completas de la tarea. El sistema demostró capacidad para completar la secuencia completa: aproximación, agarre y desmolde (ver Figura 5.3).

5.4.1. Fase 1: Aproximación y Generalización

El uso de DMPs permitió que el robot alcanzara el objeto de forma suave, separando el movimiento en fases diferenciadas para asemejarse al movimiento biológico humano. A diferencia de una interpolación lineal simple, el robot replicó la curvatura de aproximación demostrada por el operario, evitando obstáculos verticales y generando perfiles de velocidad en forma de campana.

5.4.2. Fase 2: Interacción y Agarre

La integración del gripper mediante el servicio `/ur_hardware_interface/set_io` mostró una sincronización correcta con la máquina de estados. El error de posicionamiento final promedio se estimó visualmente en < 1 cm, suficiente para realizar un agarre de fuerza sobre la pieza cilíndrica.

5.4.3. Fase 3: Manipulación (Desmolde)

Gracias a la estrategia de desacople cinemático descrita en la Sección 5.2.2, el robot ejecutó el movimiento de rosca necesario para separar la cabeza de la muñeca del cuerpo.



FIGURA 5.3: Secuencia de ejecución en el robot real UR3e. (A - Izquierda arriba) Posición inicial de reposo. (B - Izquierda abajo) Aproximación guiada por DMP. (C - Derecha arriba) Agarre del objeto mediante activación de E/S digitales. (D - Derecha abajo) Ejecución de la rotación pura de muñeca para el desmolde y separación de la cabeza.

5.5. Discusión y Limitaciones

Aunque el sistema logró completar la tarea, se identificaron limitaciones inherentes al enfoque de visión pasiva:

- **Latencia y Lazo Abierto:** El pipeline completo introduce una separación temporal que impide la corrección de errores en tiempo real si el objeto se mueve bruscamente durante la ejecución. El robot no se adapta dinámicamente durante la trayectoria ("lazo abierto"), sino que ejecuta la primitiva aprendida.
- **Dependencia de la Calibración:** La precisión del agarre depende críticamente de la calibración estéreo y la calibración mano-ojo (T_{cam}^{base}). Pequeñas desviaciones físicas en las cámaras se traducen en errores de agarre.

No obstante, los experimentos validan la hipótesis principal: es posible transferir habilidades de manipulación complejas (como desenroscar) a un robot colaborativo

utilizando únicamente observación visual y algoritmos de aprendizaje por imitación, sin necesidad de programación explícita de coordenadas.

6 Conclusiones y Líneas de Trabajo Futuro

El presente Trabajo de Fin de Grado ha abordado el diseño, implementación y validación de un sistema completo de Aprendizaje por Demostración (LfD) para un brazo robótico colaborativo UR3e. El objetivo principal, dotar al robot de la capacidad de aprender tareas de manipulación complejas a partir de la observación visual pasiva de un operario humano, se ha alcanzado satisfactoriamente.

A continuación, se detallan las conclusiones extraídas del desarrollo y las propuestas para la continuidad de la investigación.

6.1. Conclusiones Generales

El sistema desarrollado demuestra que es posible transferir habilidades motoras del humano al robot sin necesidad de instrumentación invasiva o compleja, ni programación explícita de puntos. Se ha validado la hipótesis de que un sistema de visión estéreo de bajo coste, combinado con algoritmos de estimación de pose modernos (MediaPipe), proporciona suficiente precisión para tareas de manipulación fina, como el desmolde de piezas, superando las barreras de entrada de la programación robótica tradicional.

6.2. Conclusiones Técnicas

6.2.1. Sobre el Sistema de Percepción

La decisión de utilizar un enfoque basado en **triangulación de puntos clave (Keypoints)** frente al procesamiento de nubes de puntos densas (Point Clouds) ha resultado ser crítica para el rendimiento. Se ha comprobado que procesar solo las 33 articulaciones del esqueleto reduce drásticamente la carga computacional, permitiendo una tasa de refresco adecuada para la captura de movimientos rápidos. Además, el uso de una configuración multi-vista ha mitigado eficazmente los problemas de oclusión inherentes a la manipulación manual.

6.2.2. Sobre el Mapeo Cinemático y Control

Uno de los hallazgos más relevantes de este trabajo es la superioridad, para esta aplicación concreta, del “mapeo geométrico directo” frente a los solvers de cinemática inversa (IK) genéricos.

- La implementación de la **reflexión morfológica** resolvió el problema de discrepancia estructural (codo humano hacia abajo vs. codo robot hacia arriba) de

forma determinista, eliminando las configuraciones peligrosas que los solvers numéricos tendían a generar.

- El desacople cinemático durante la fase de rotación (bloqueo de articulaciones base) demostró ser una estrategia eficaz para evitar movimientos parásitos en robots redundantes o singulares.

6.2.3. Sobre la Generalización con DMPs

El uso de Primitivas de Movimiento Dinámico (DMPs) ha permitido transformar grabaciones ruidosas y variables en trayectorias robóticas suaves y seguras. El sistema no se limita a “repetir” lo grabado, sino que extrae la intención del movimiento (la forma), permitiendo adaptar la ejecución a nuevos puntos de inicio o fin sin re-entrenar, lo cual aporta la flexibilidad necesaria para entornos industriales dinámicos.

6.3. Líneas de Trabajo Futuro

A pesar de los resultados positivos, el sistema presenta limitaciones que abren la puerta a futuras mejoras dentro del proyecto ROBOTA-SUDOE:

- **Integración de Control de Fuerza:** El UR3e posee sensores de par en las articulaciones. Una mejora vital sería utilizar estos datos para detener el robot o ajustar la fuerza de agarre si se detecta resistencia anómala (ej. si la rosca de la muñeca se atasca), aumentando la seguridad de la operación. Adicionalmente, se podría enriquecer la demostración humana incorporando sensórica IMU o guantes de datos para capturar el perfil de fuerza ejercido durante el desmolde.
- **Generalización a Tareas Bimanuales:** El pipeline de visión ya detecta ambas manos. Una extensión natural sería utilizar toda la información disponible para coordinar dos brazos robóticos: uno encargado de sujetar la pieza (rol de fijación) y el otro de manipularla (rol activo), o en su defecto, diseñar un anclaje activo sincronizado con el robot.
- **Migración a ROS 2:** Para garantizar la compatibilidad con los estándares modernos de la industria y mejorar la gestión del tiempo real, se plantea la migración de todo el stack de software a ROS 2 (Humble/Jazzy).
- **Ampliación del Dataset y Generalización:** Actualmente se podría mejorar la precisión del aprendizaje usando un dataset más amplio, permitiendo una generalización más robusta ante variaciones espaciales. Asimismo, se propone extrapolar este aprendizaje a otras tareas de manipulación más allá del desmolde, validando la versatilidad del pipeline.
- **Control en Lazo Cerrado (Visual Servoing):** Aunque las DMPs ofrecen una excelente generalización de la trayectoria, la ejecución actual es en lazo abierto. Se propone incorporar un bucle de corrección en tiempo real basado en visión artificial (*Visual Servoing*) para ajustar la posición final del efector ante movimientos imprevistos del objeto durante la fase de aproximación.

A Repositorio de Código y Recursos Digitales

El desarrollo completo de este Trabajo de Fin de Grado, incluyendo los scripts de visión, algoritmos de control, notebooks de entrenamiento y datasets, se encuentra disponible en acceso abierto en el siguiente repositorio de GitHub:

https://github.com/Sergy117/TFG_LearningByDemo_Ur3e

A continuación, se describe la estructura de directorios del repositorio para facilitar la reproducibilidad de los experimentos:

- /auxiliary_tools
 - /YOLO_train: Scripts y configuración para el entrenamiento de la red neuronal.
 - README.md: Documentación de la metodología de entrenamiento.
 - cap.py: Script de captura de imágenes para la creación del dataset.
 - data.yaml: Archivo de configuración de YOLO con las rutas relativas.
 - requirements.txt: Lista de dependencias de Python necesarias.
 - retrain_doll_yolo.py: Script para el re-entrenamiento (Fine-tuning).
 - test_onnx_inference.py: Script de validación para inferencia en formato ONNX.
 - train_doll_yolo.py: Script principal de entrenamiento.
 - yolo8n.pt: Pesos del modelo YOLOv8 nano pre-entrenado.
 - /cam_cal: Herramientas de calibración estéreo.
 - /data: Archivos .npy con las matrices extrínsecas e intrínsecas (T , R , K).
 - /images: Dataset de imágenes del tablero de ajedrez capturadas.
 - README.md: Instrucciones de calibración.
 - CalibrationStereo.py: Script que procesa las imágenes del tablero y genera las matrices de calibración.
- /vision_system: Pipeline de percepción en tiempo real.
 - vision_utils.py: Librería de funciones auxiliares y clases de procesamiento.

- `main_script_grabacion.py`: Script principal para la captura y grabación de demostraciones.
- `README.md`: Documentación de uso.
- `requirements.txt`: Dependencias específicas del módulo de visión.
- `/robot_control`: Algoritmos de mapeo, aprendizaje y ejecución.
 - `/ws_moveit`: Espacio de trabajo de ROS (Catkin Workspace) que contiene los paquetes de control del UR3e real y simulado, así como configuraciones de RViz.
 - `README.md`: Instrucciones para el despliegue en ROS Noetic.
- `/data/learned_dmps`: Archivos serializados (`.pk1`) que contienen los modelos DMP entrenados listos para ejecución.

Todos los directorios incluyen archivos `README.md` específicos con instrucciones detalladas de instalación y ejecución para facilitar la réplica de los resultados.

Bibliografía

- [1] B. D. Argall, S. Chernova, M. Veloso y B. Browning, «A survey of robot learning from demonstration,» *Robotics and autonomous systems*, vol. 57, n.º 5, págs. 469-483, 2009.
- [2] C. L. Nehaniv y K. Dautenhahn, «The correspondence problem,» en *Imitation in animals and artifacts*, MIT Press, 2002, págs. 41-61.
- [3] A. Billard, S. Calinon, R. Dillmann y S. Schaal, «Robot programming by demonstration,» en *Springer handbook of robotics*, Springer, 2016, págs. 1371-1394.
- [4] Universal Robots, *UR3e User Manual*, Version 5.12, Universal Robots A/S, Odense, Denmark, 2023.
- [5] A. García-Pérez y Otros, «Análisis cinemático del robot UR3 mediante la metodología de Denavit-Hartenberg,» *Pistas Educativas*, vol. 45, n.º 144, 2023. dirección: <https://pistaseducativas.celaya.tecnm.mx/index.php/pistas/article/view/2957>.
- [6] Intel Realsense: Github library, <https://github.com/realsenseai/librealsense>, 2023.
- [7] Z. Zhang, «A flexible new technique for camera calibration,» *IEEE Transactions on pattern analysis and machine intelligence*, vol. 22, n.º 11, págs. 1330-1334, 2000.
- [8] V. Bazarevsky, I. Grishchenko, K. Raveendran, T. Zhu, F. Zhang y M. Grundmann, «Blazepose: On-device real-time body pose tracking,» en *arXiv preprint arXiv:2006.10204*, 2020.
- [9] J. Randika, K. Rajamanthri et al., «Motion Capturing in cricket with bare minimum hardware and optimised software: A comparison of MediaPipe and OpenPose,» en *2024 International Conference on Social Sustainable and Innovative Computing (SSITCON)*, 2024, págs. 1-8. DOI: [10.1109/SSITCON62437.2024.10796169](https://doi.org/10.1109/SSITCON62437.2024.10796169).
- [10] R. Hartley y A. Zisserman, *Multiple view geometry in computer vision*. Cambridge university press, 2003.
- [11] T. Oberlies, T. Hanselmann, M. Schwaiger y O. Von Stryk, «Object localization and performance evaluation in the RoboCup four-legged league,» en *Proceedings of the 1st International Conference on Simulation, Modeling, and Programming for Autonomous Robots (SIMPAN)*, Venice, Italy, 2008, págs. 1-12.
- [12] Y. LeCun, Y. Bengio y G. Hinton, «Deep learning,» *nature*, vol. 521, n.º 7553, págs. 436-444, 2015.
- [13] MatLabs, *CNN*, <https://la.mathworks.com/discovery/convolutional-neural-network.html>, 2025.
- [14] J. Redmon, S. Divvala, R. Girshick y A. Farhadi, «You only look once: Unified, real-time object detection,» en *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, págs. 779-788.
- [15] G. Jocher, A. Chaurasia y J. Qiu, *Ultralytics YOLO*, ver. 8.0.0, GitHub repository, 2023. dirección: <https://github.com/ultralytics/ultralytics>.

- [16] A. J. Ijspeert, J. Nakanishi, H. Hoffmann, P. Pastor y S. Schaal, «Dynamical movement primitives: learning attractor models for motor behaviors,» *Neural computation*, vol. 25, n.º 2, págs. 328-373, 2013.
- [17] T. DeWolf, *Dynamic Movement Primitives Part 1: The Basics*, <https://studywolf.wordpress.com/>, Accedido: 2024-05-20, 2013.
- [18] T. Y. Yevgen Chebotar, RT2, <https://deepmind.google/blog/rt-2-new-model-translates-vision-and-language-into-action/>, 2023.
- [19] Vicon, *Vicon*, <https://www.vicon.com/>, 2023.
- [20] NaturalPoint, *OptiTrack*, <https://optitrack.com/>, 2023.
- [21] A. Chatzitofis, D. Zarpalas, P. Daras y S. Kollias, «DeMoCap: Low-Cost Marker-Based Motion Capture,» *International Journal of Computer Vision*, vol. 129, dic. de 2021. DOI: 10.1007/s11263-021-01526-z.
- [22] ONNX, *ONNX*, <https://onnx.ai/>, 2025.
- [23] OpenCV, *OpenCV*, <https://opencv.org/>, 2025.
- [24] S. González, *Learning and Validation*, https://github.com/Sergy117/TFG_LearningByDemo_Ur3e/tree/main/learning_and_validation, 2025.
- [25] R. W. Schafer, «What is a Savitzky-Golay filter?» *IEEE Signal processing magazine*, vol. 28, n.º 4, págs. 111-117, 2011.
- [26] S. Khosla, *Locally Weighted Linear Regression*, <https://www.geeksforgeeks.org/machine-learning/ml-locally-weighted-linear-regression/>, 2025.